
Chapter 7

Trees

Trees

Let A be a set, and let T be a relation on A .

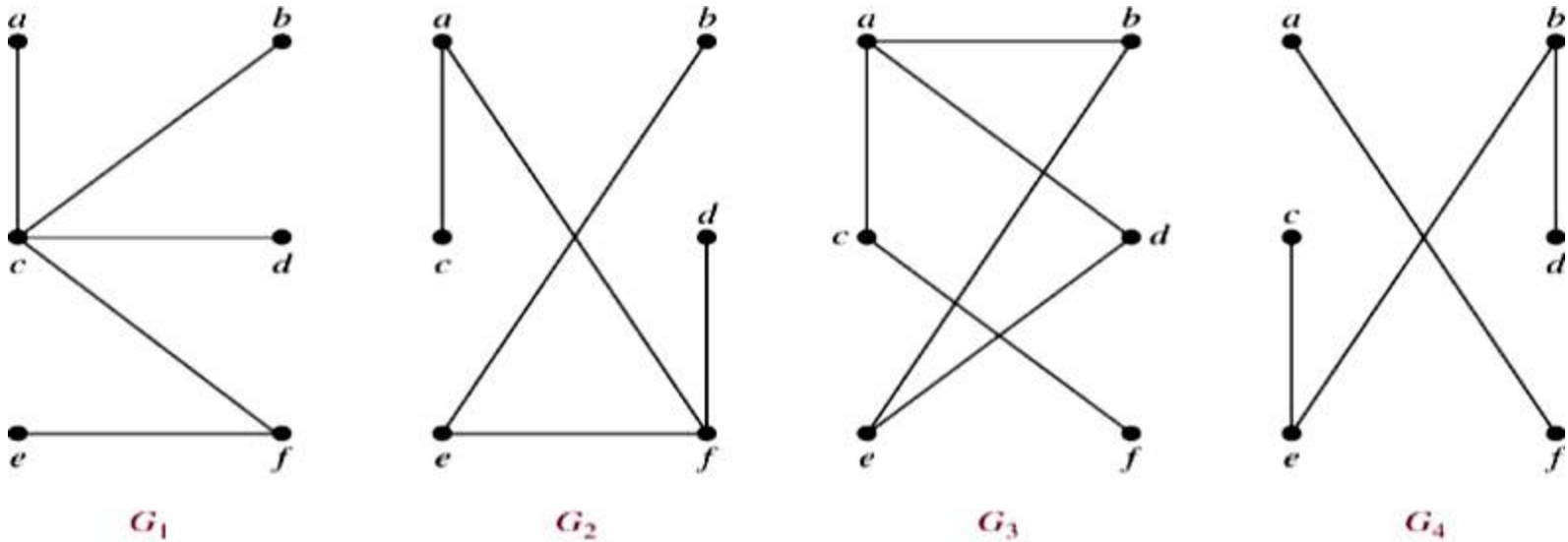
We say that T is a tree if there is a vertex x in A with the property that

- There is **a unique path** in T from x to every other vertex in A .
- No path from x to x .

Usually, x is called the root of the tree, and T is referred to as a rooted tree (denoted by (T, x))

Def. A **tree** is a connected undirected graph with no simple circuits.

Example Which of the graphs are trees?



Sol: G_1, G_2

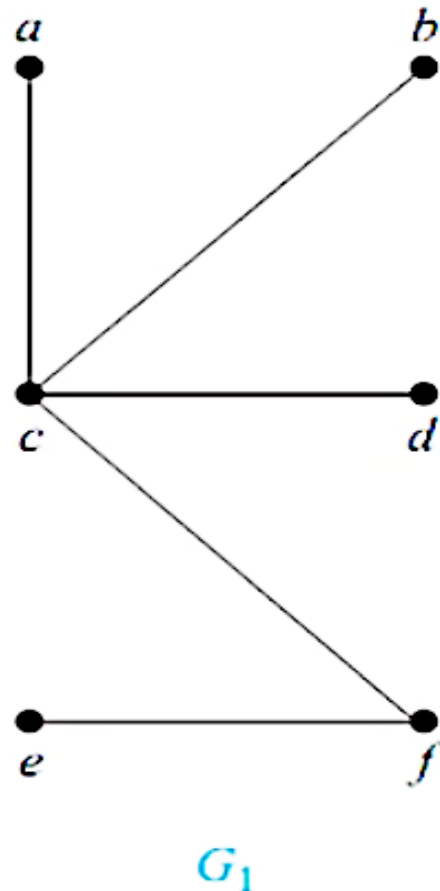
Note. If connected condition is removed, it becomes forest

Example 1

Example: Determine if the following graph is a tree?

- Connected : **Yes**
- Has Circuit : **No**

$\therefore G_1$ is a tree.

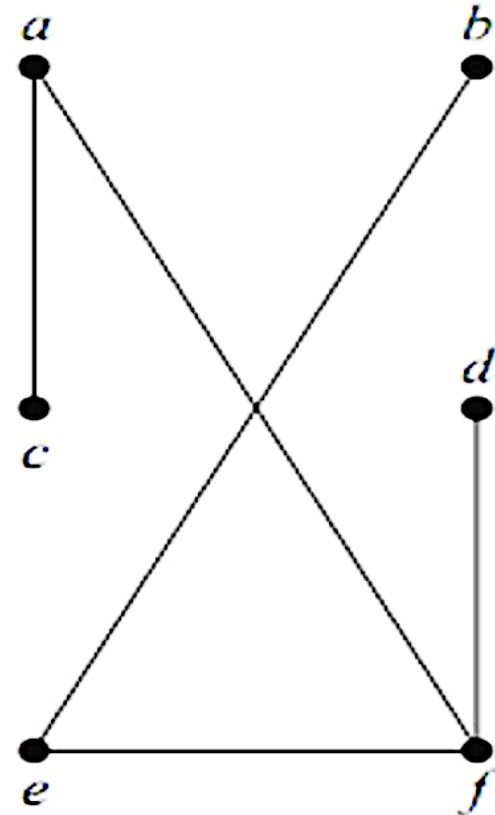


Example 2

Example: Determine if the following graph is a tree?

- Connected : **Yes**
- Has Circuit : **No**

$\therefore G_2$ is a tree.



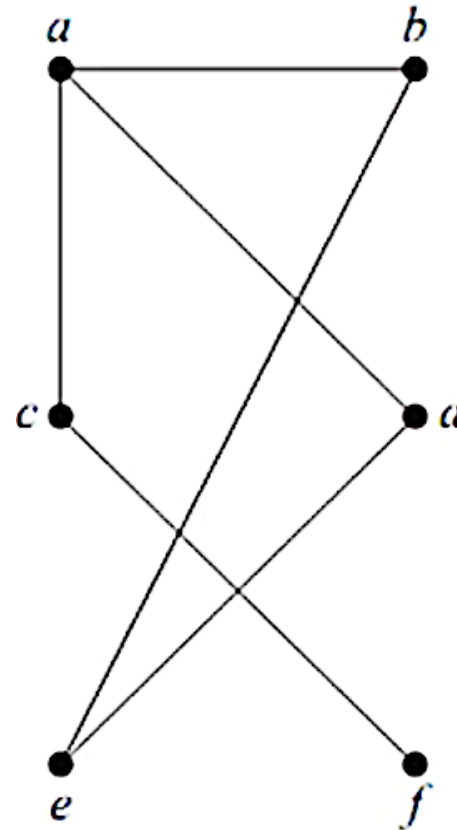
G_2

Example 3

Example: Determine if the following graph is a tree?

- Connected : **Yes**
- Has Circuit : **Yes**

$\therefore G_3$ is **NOT** a tree.



G_3

Example 4

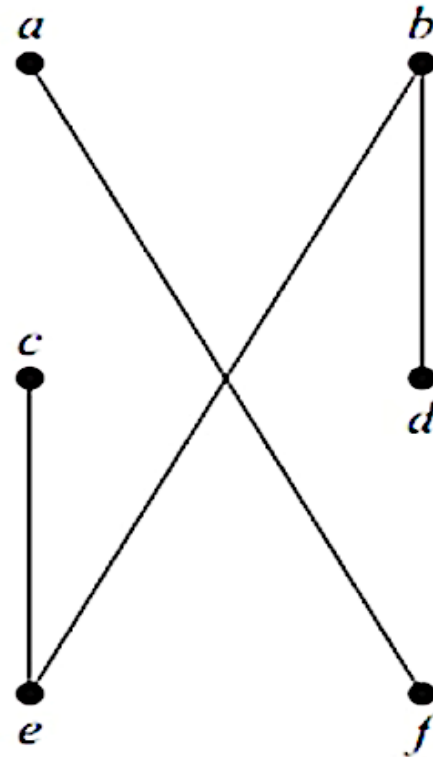
Example: Determine if the following graph is a tree?

- Connected : **No**

$\therefore G_4$ is **NOT** a tree.

•

It is a **forest** if the graph has more than one tree

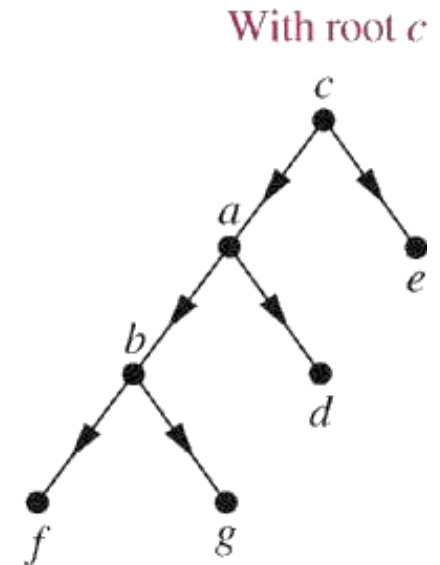
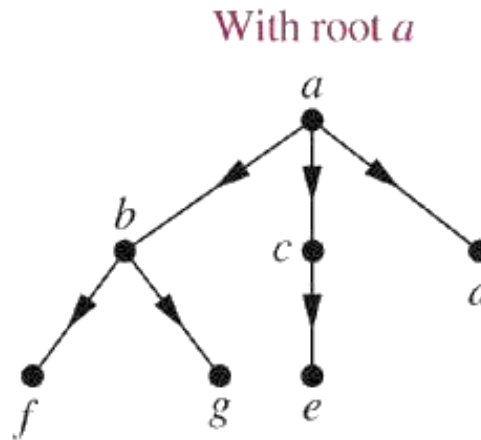
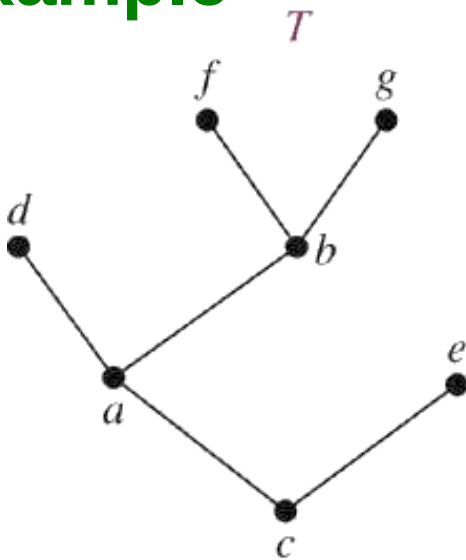


G_4

Any undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.

Def 2. A **rooted tree** is a tree in which one vertex has been designed as the root and every edge is directed away from the root. (Arrow to disappear)

Example



Trees

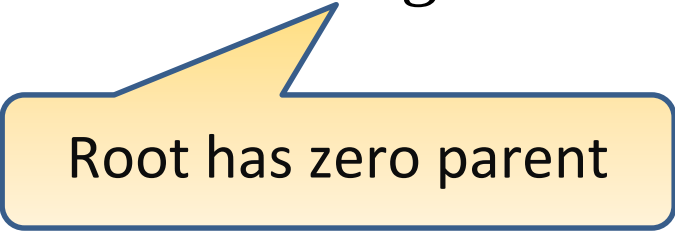
■ Theorem 1

Let (T, x) be a **rooted tree**. Then

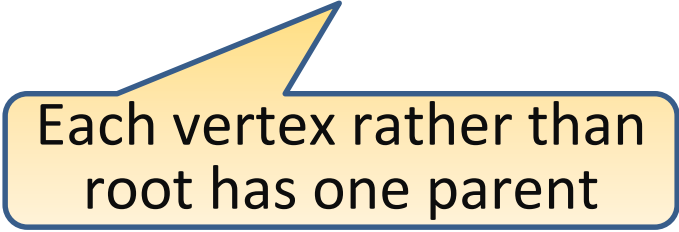
(a) There are no cycle in T

(b) x is the only root of T

(c) Each vertex in T , other than x , has in-degree one, and x has in-degree zero.



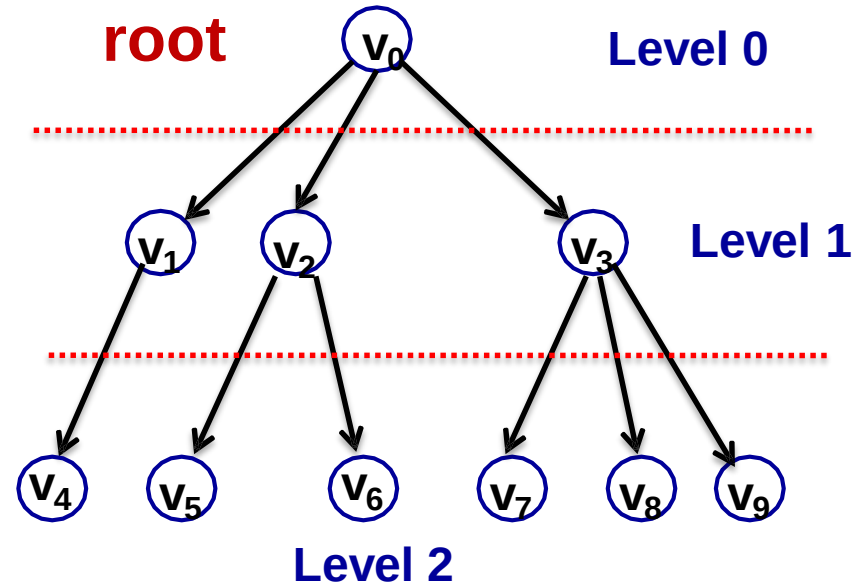
Root has zero parent



Each vertex rather than root has one parent

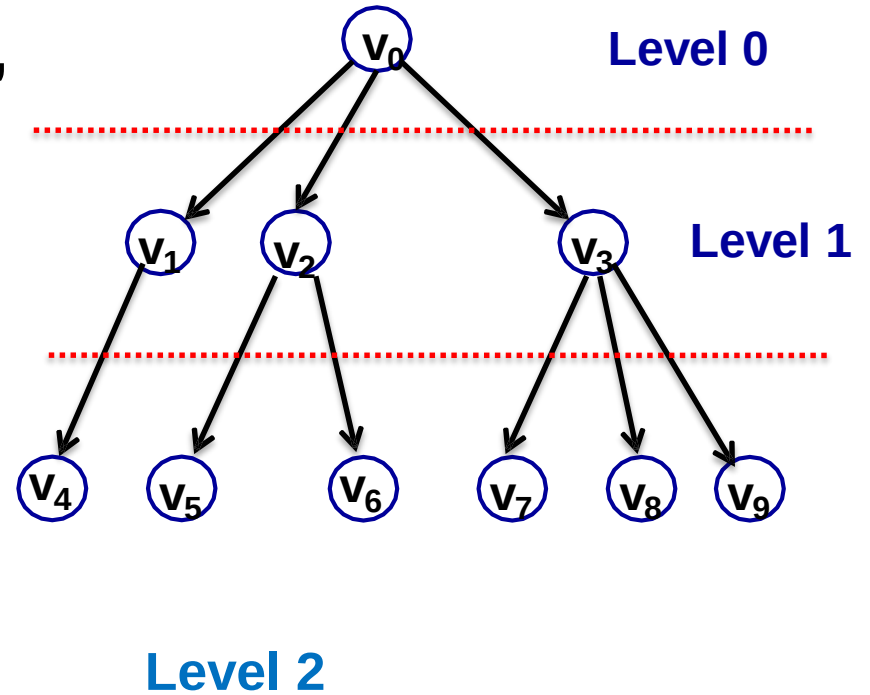
Trees

- **Level 0 vertices**: the vertices at level 0.
- **Level k vertices**: the vertices of k level.
- Example: level 2 vertices: $v_4, v_5, v_6, v_7, v_8, v_9$
- **Height of a tree**: the largest level number of a tree. Here, $\text{height}=2$.



Relation as a tree

$R = \{(v_0, v_1), (v_0, v_2), (v_0, v_3),$
 $(v_1, v_4), (v_2, v_5), (v_2, v_6),$
 $(v_3, v_7), (v_3, v_8), (v_3, v_9)\}$



Trees

- **Parent-offspring:**

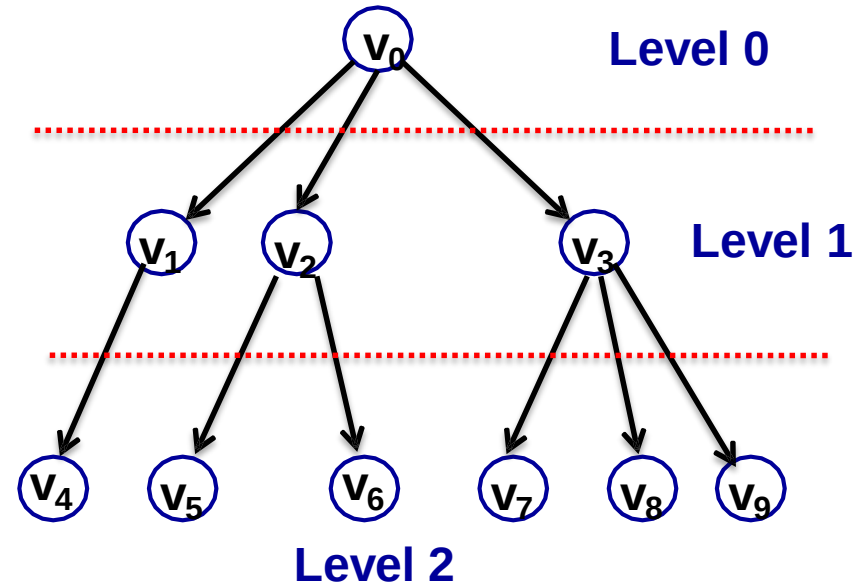
For all pairs (a, b) in T , a is called the parent of b , and b is called the offspring of a .

- **Siblings:**

The vertices that have the same parent

- **Leaves:**

The vertices that have no children.



- **internal vertices:**

Vertices that have children.

Example

level

0

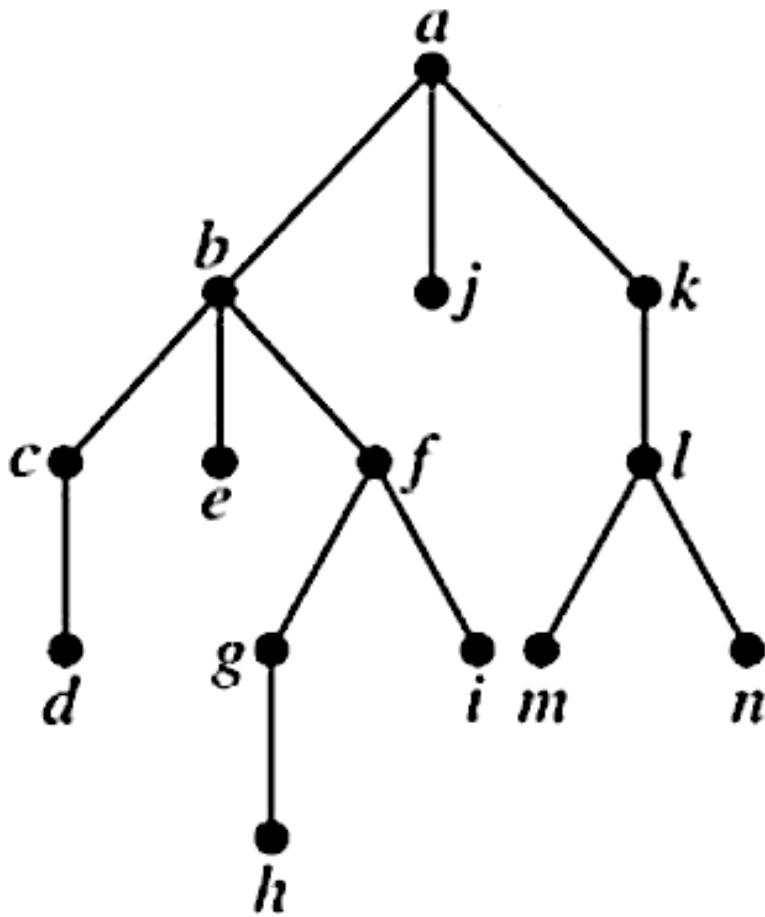
1

2

3

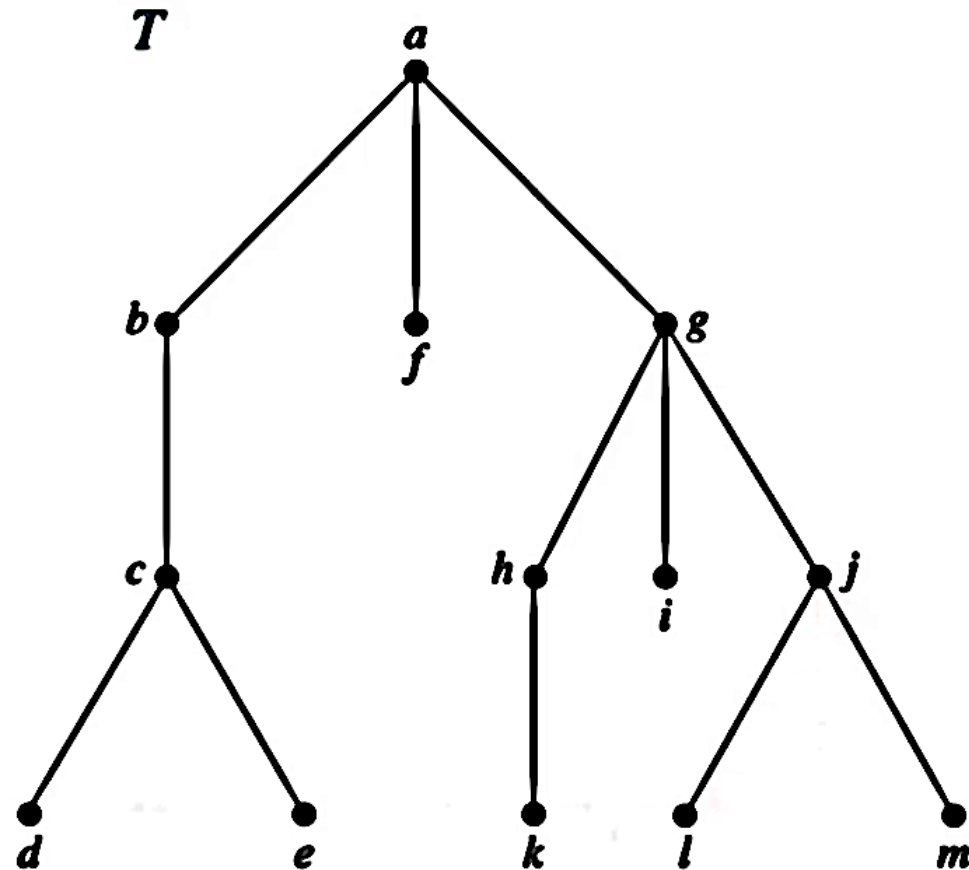
4

height = 4

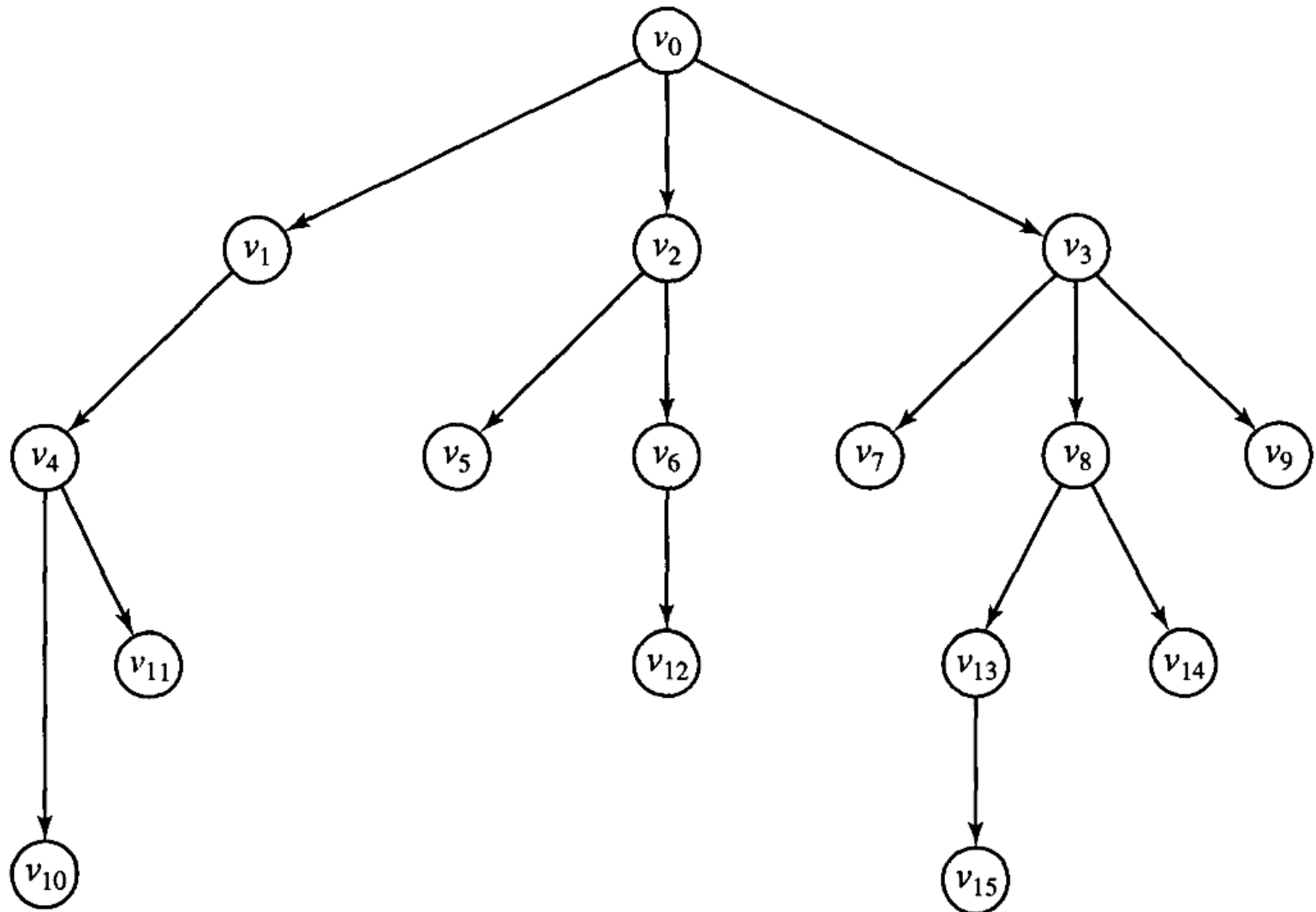


Rooted tree

- Find parent of c.
 $\{b\}$
 - Find children of g.
 $\{h, i, j\}$
 - Find siblings of h.
 $\{i, j\}$
 - Find ancestors of e.
 $\{c, b, a\}$
 - Find descendants of b.
 $\{c, d, e\}$
 - Find all internal nodes.
 $\{a, b, g, c, h, j\}$
 - Find all leaves (external nodes).
 $\{f, d, e, i, k, l, m\}$
-



Question



(T, v_0)

Answer the following

List all level-3 vertices?

v10, v11, v12, v13, v14

List all leaves?

v10, v11, v5, v12, v7, v15, v14, v9

What are siblings of v8?

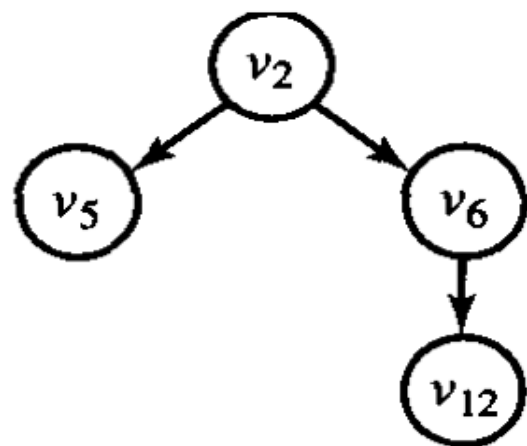
v7, v9

What are descendants of v8?

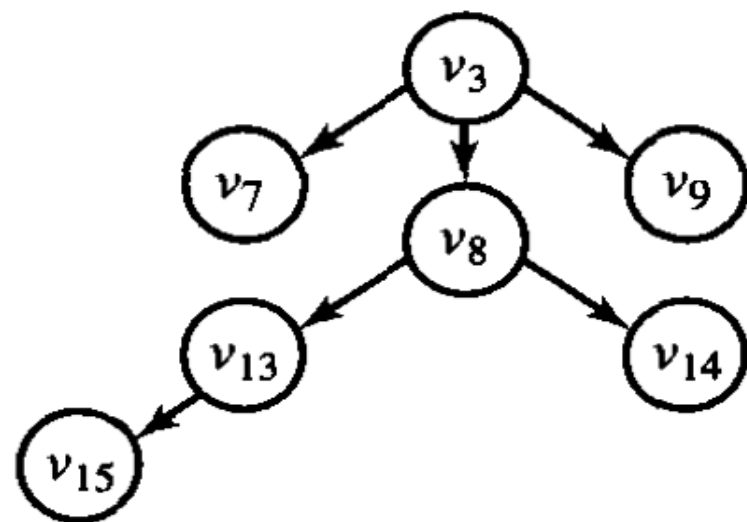
v13, v14, v15

-
- (a) Compute the tree $T(v_2)$.
(b) Compute the tree $T(v_3)$.

(a)



(b)



-
- (a) What is the height of (T, v_0) ? a) 4
(b) What is the height of $T(v_3)$? b) 3

Is (T, v_0) an n -tree? If so, for what integer n ?
Is (T, v_0) a complete n -tree? If so, for what integer n ?

Sol.

(T, v_0) may be an n -tree for $n \geq 3$. It is not a complete 3-tree.

Theorem

Let (T, x) be a rooted tree on a set A . Then

(a) T is irreflexive

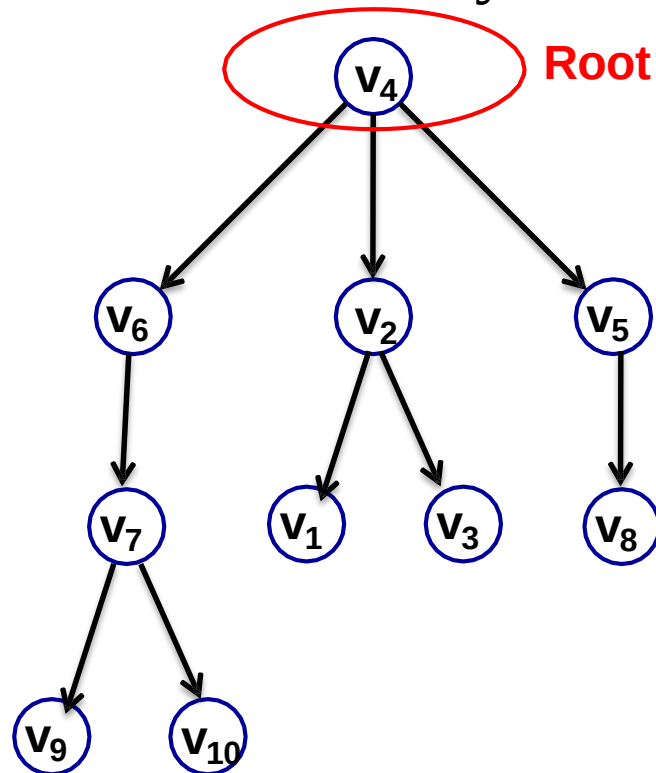
(b) T is asymmetric

(c) If (a,b) in T and (b,c) in T , then (a,c) is not in T , for all a, b and c in A . $\rightarrow$ not transitive

Trees

■ Example

Let $A = \{v_1, v_2, \dots, v_{10}\}$ and let $T = \{(v_2, v_3), (v_2, v_1), (v_4, v_5), (v_4, v_6), (v_5, v_8), (v_6, v_7), (v_4, v_2), (v_7, v_9), (v_7, v_{10})\}$. Show that T is a rooted tree and identify the root.



In Exercises 1 through 8, each relation R is defined on the set A . In each case determine if R is a tree and, if it is, find the root.

1. $A = \{a, b, c, d, e\}$
 $R = \{(a, d), (b, c), (c, a), (d, e)\}$

1. Yes, the root is b .

3. Yes, the root is f .

2. $A = \{a, b, c, d, e\}$
 $R = \{(a, b), (b, e), (c, d), (d, b), (c, a)\}$

5. No.

3. $A = \{a, b, c, d, e, f\}$
 $R = \{(a, b), (c, e), (f, a), (f, c), (f, d)\}$

Irreflexive,
 asymmetric, not
 transitive as child
 can't be a parent
 of its parent.

4. $A = \{1, 2, 3, 4, 5, 6\}$
 $R = \{(2, 1), (3, 4), (5, 2), (6, 5), (6, 3)\}$

5. $A = \{1, 2, 3, 4, 5, 6\}$
 $R = \{(1, 1), (2, 1), (2, 3), (3, 4), (4, 5), (4, 6)\}$

6. $A = \{1, 2, 3, 4, 5, 6\}$
 $R = \{(1, 2), (1, 3), (4, 5), (4, 6)\}$

Trees

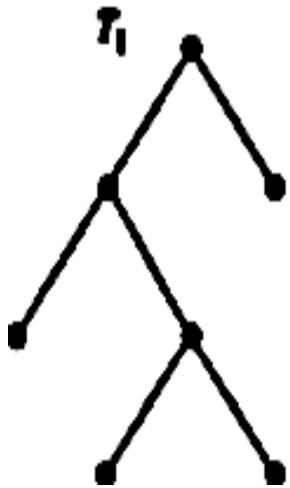
- **n-tree**

If n is a positive integer, we say that a tree is an n-tree if every vertex has at most n offspring. In particular, a 2-tree is called a binary tree.

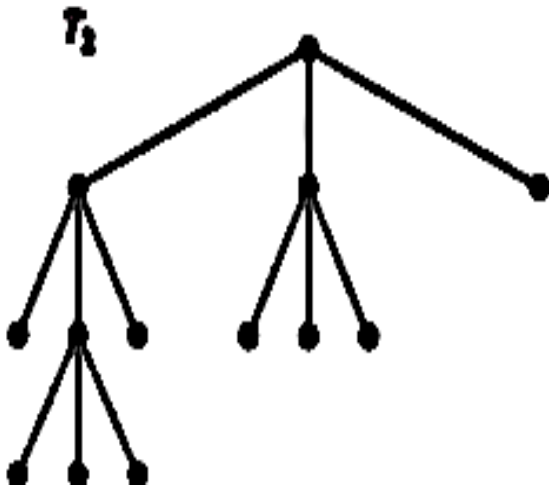
- **Complete n-tree**

If all vertices of T , other than the leaves, have exactly n offspring, we say that T is a complete n -tree. A complete 2-tree is called a completed binary tree.

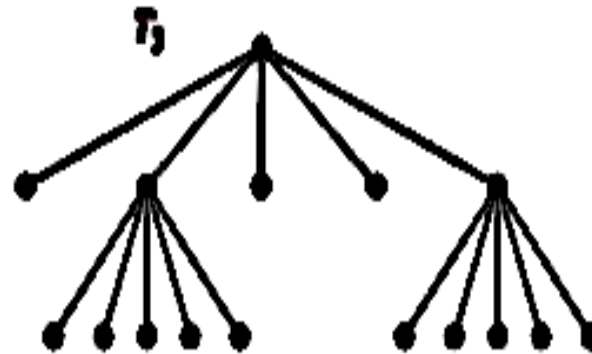
Example



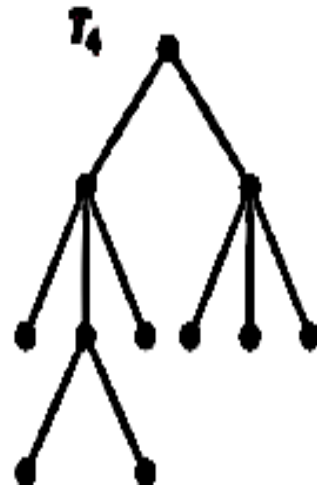
Complete binary tree



Complete 3- tree

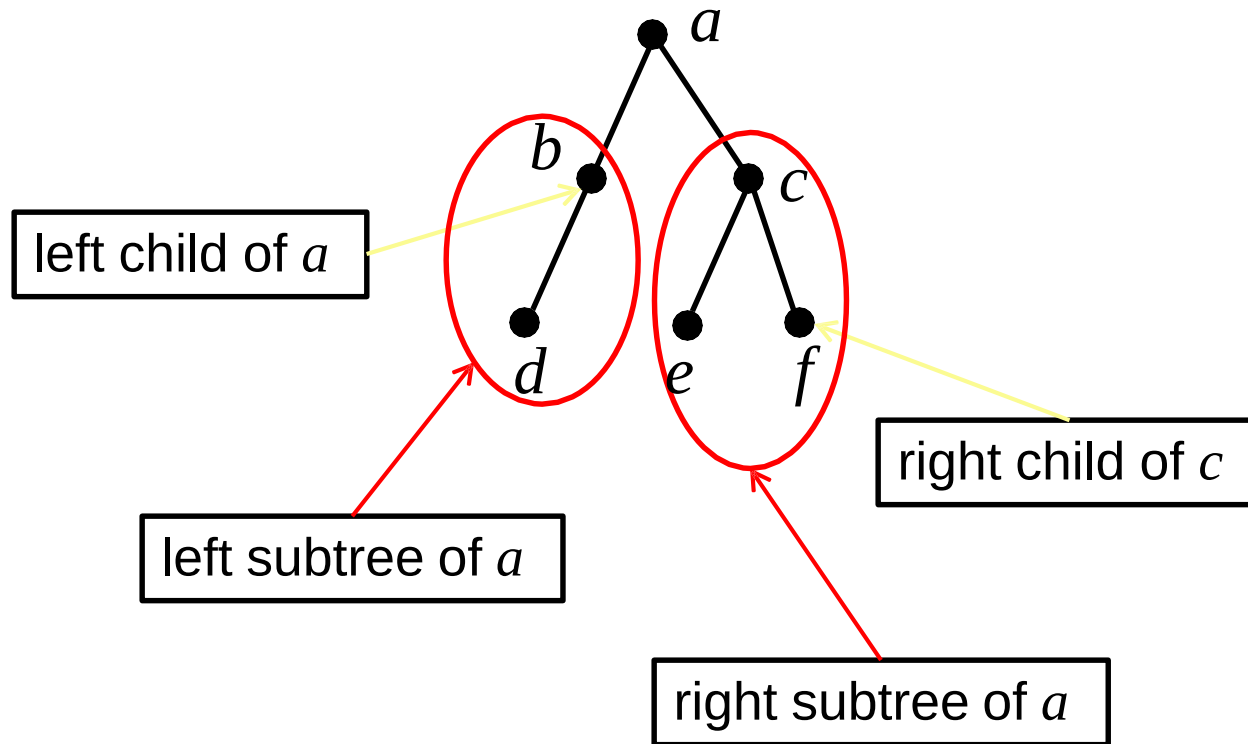


Complete 5- tree



not complete 3-tree

Def:



Properties of Trees

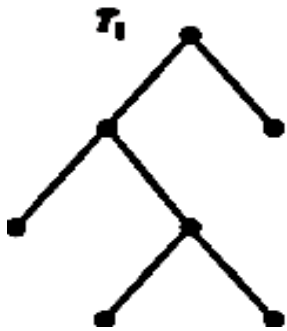
A tree with n vertices has $n-1$ edges.

A complete m -tree with i internal vertices contains $n = mi + 1$ vertices.

A complete m -tree with n vertices contains $(n-1)/m$ internal vertices, and hence $((m-1)n+1)/m$ leaves.

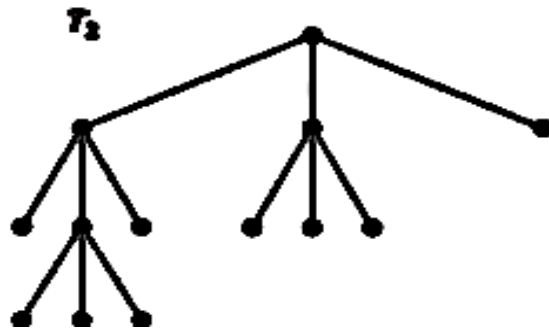
Example

A complete m -tree with i internal vertices contains $n = mi + 1$ vertices.



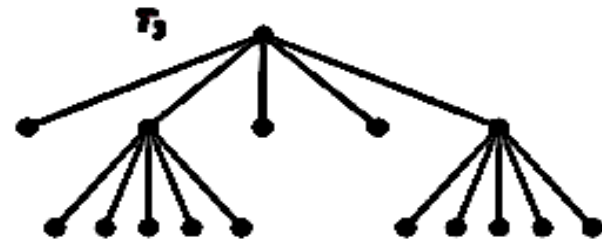
Complete binary tree

$$n = 2 \cdot 3 + 1 = 7$$



Complete 3- tree

$$n = 3 \cdot 4 + 1 = 13$$



Complete 5- tree

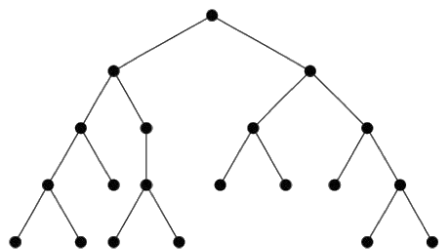
$$n = 5 \cdot 3 + 1 = 16$$

- A a complete 3-tree with 4 internal vertices has a number of vertices equal to 13.
- A a complete 5-tree with 3 internal vertices has a number of vertices equal to 16.

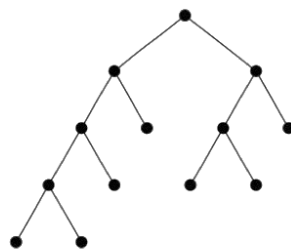
Balanced tree

A rooted tree of height h is *balanced* if all leaves are at levels h or $h - 1$.

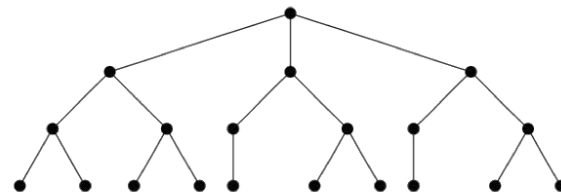
Example: Which of the rooted trees shown below is balanced?



T_1



T_2



T_3

Solution: T_1 and T_3 are balanced, but T_2 is not because it has leaves at levels 2, 3, and 4.

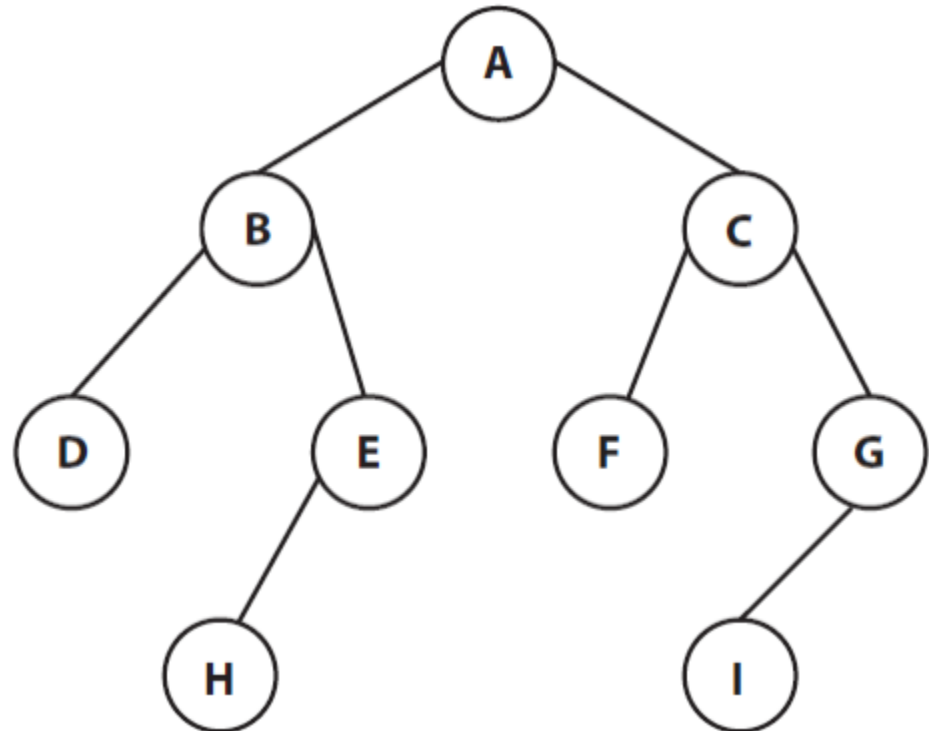
Traversing with Tree

- ❑ The tree traversing is the way to visit all the nodes of the tree on a specific order.
 - ❑ The Tree traversal can be accomplished in four different ways:
 - ❑ Inorder Traversal
 - ❑ Pre Order Traversal.
 - ❑ Post Order Traversal.
-

Inorder Traversal

- ❑ Traverse the Left Subtree in INORDER(Left)
- ❑ Visit the Root node
- ❑ Traverse the Right Subtree in INORDER(Right)

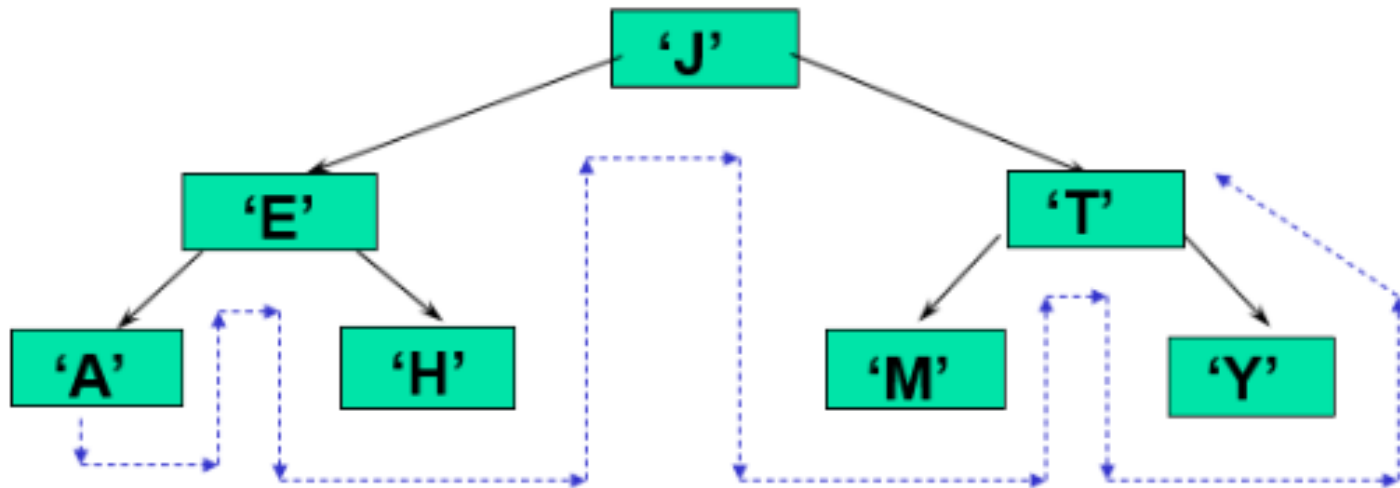
Inorder : **D B H E A F C I G**



Inorder Traversal



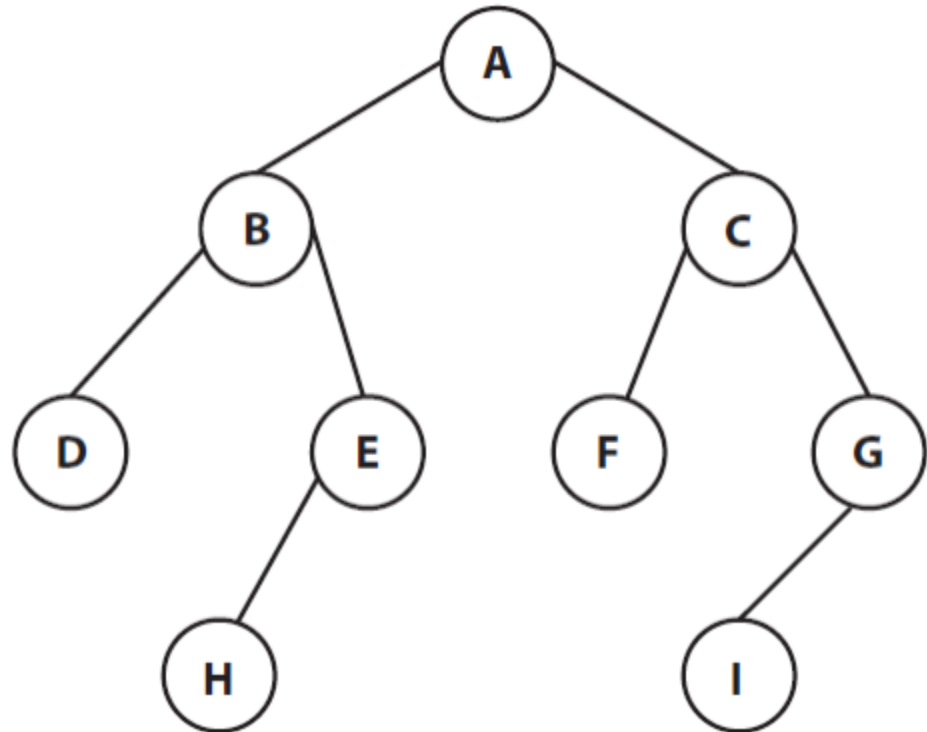
Inorder Traversal: A E H J M T Y



Pre Order Traversal

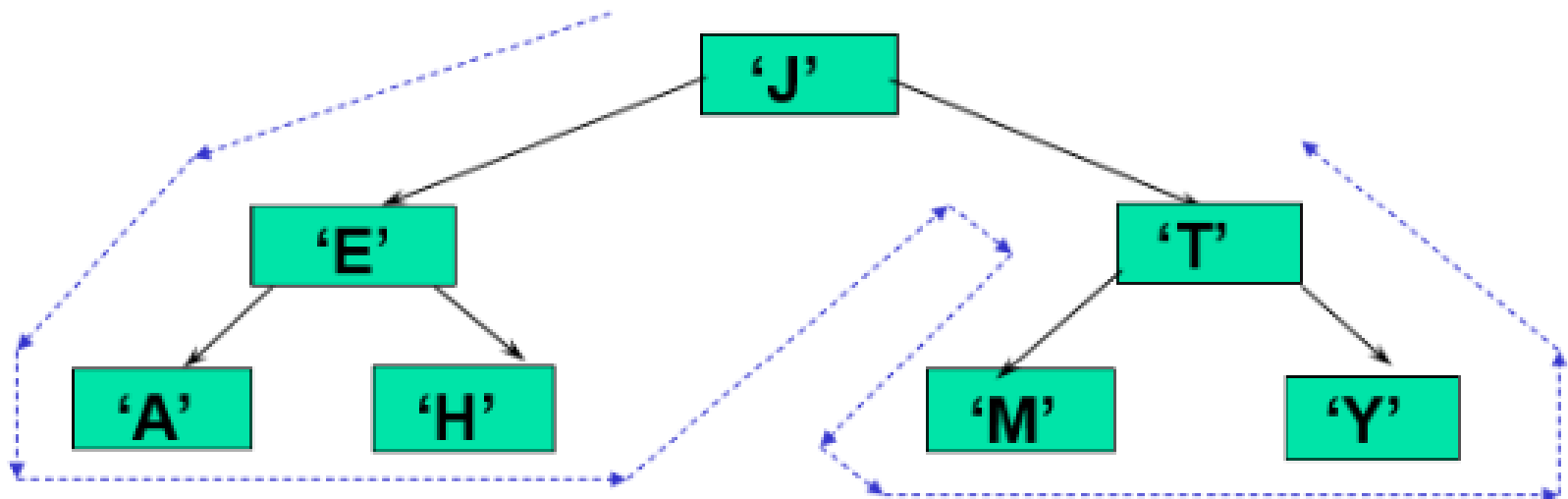
- ❑ Visit the Root Node
- ❑ Traverse the Left Subtree in PREORDER(Left)
- ❑ Traverse the Right Subtree in PREORDER(Right)

Preorder : **A B D E H C F G I**



Pre Order Traversal

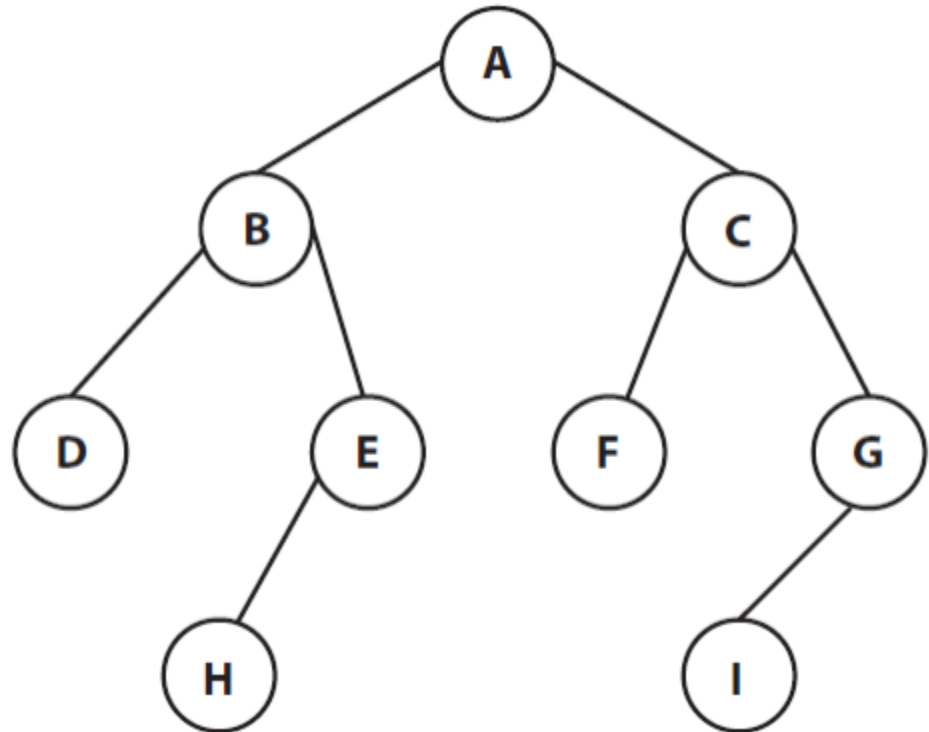
Preorder Traversal Example: J E A H T M Y



Post Order Traversal

- ❑ Traverse the Left Subtree in POSTORDER(Left)
- ❑ Traverse the Right Subtree in POSTORDER(Right)
- ❑ Visit the Root Node

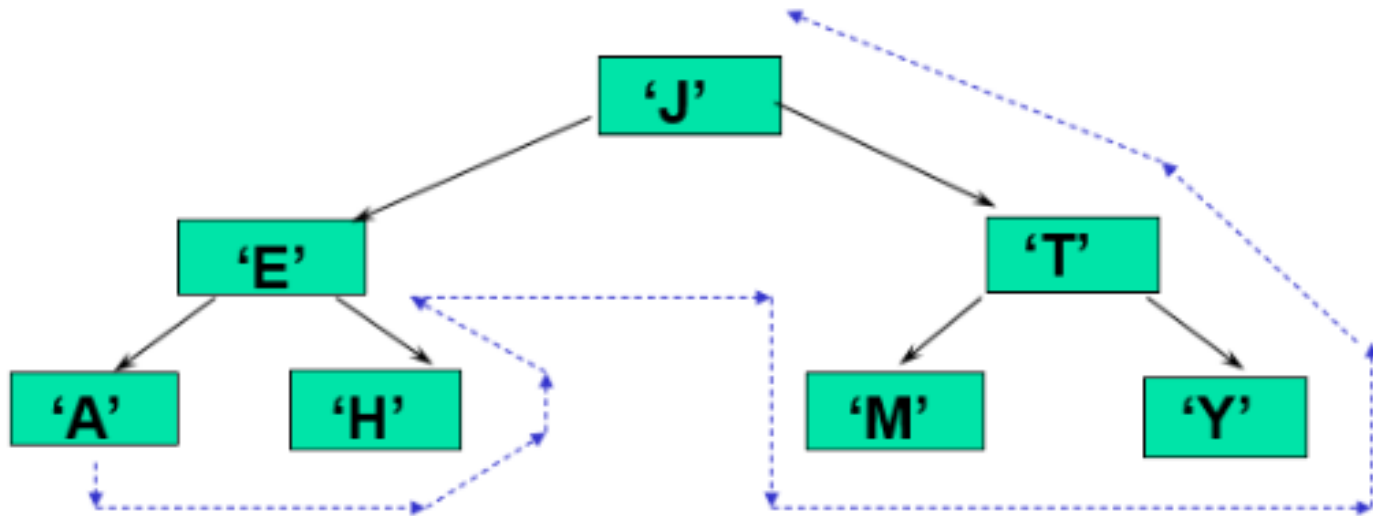
Post Order : **D H E B F I G C A**



Post Order Traversal



Postorder Traversal: A H E M Y T J



Expression Tree

- Use binary tree to denote the following algebraic expression
- $(3 - (2 * x)) + ((x - 2) - (3 + x))$

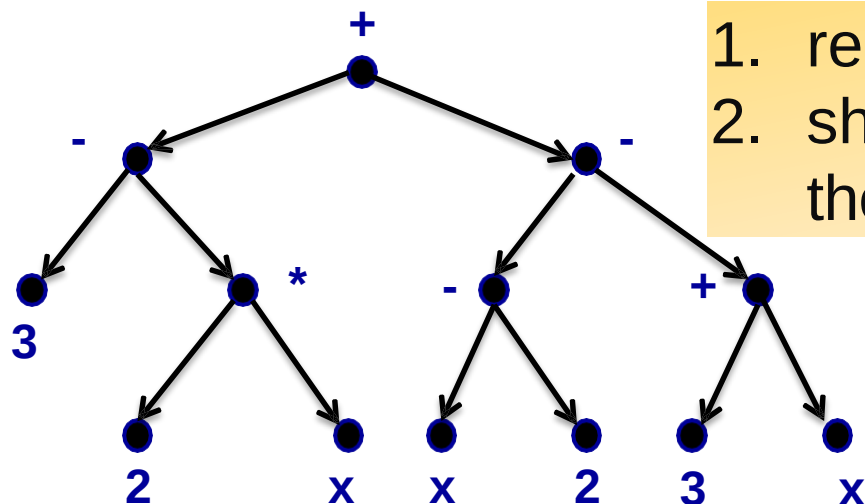
()

^ ↑

* /

+ -

=

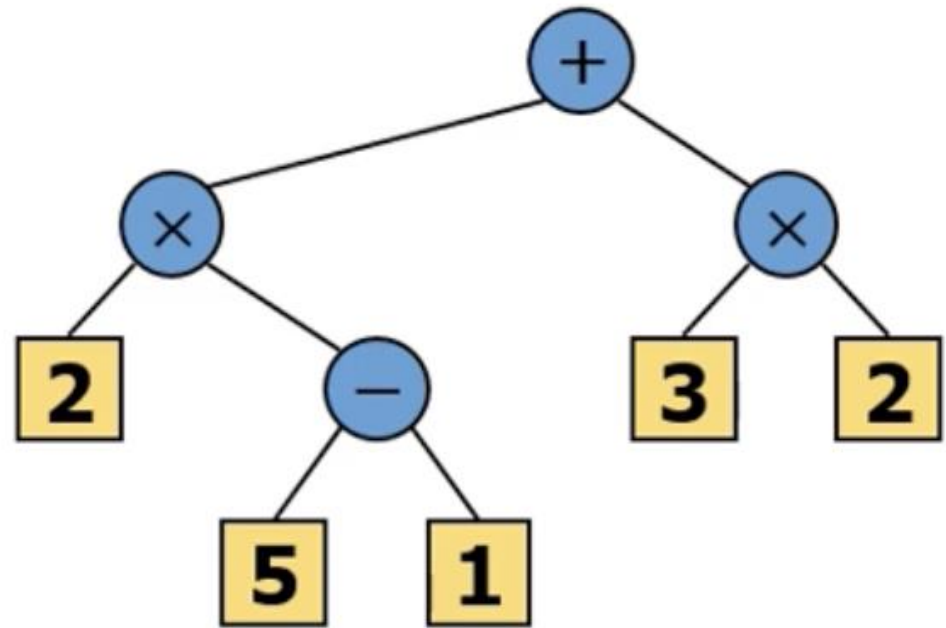


1. represent the vertices simply as dots
2. show the label of each vertex next to the dot representing that vertex

Here leaves denote the numbers or variables

Example

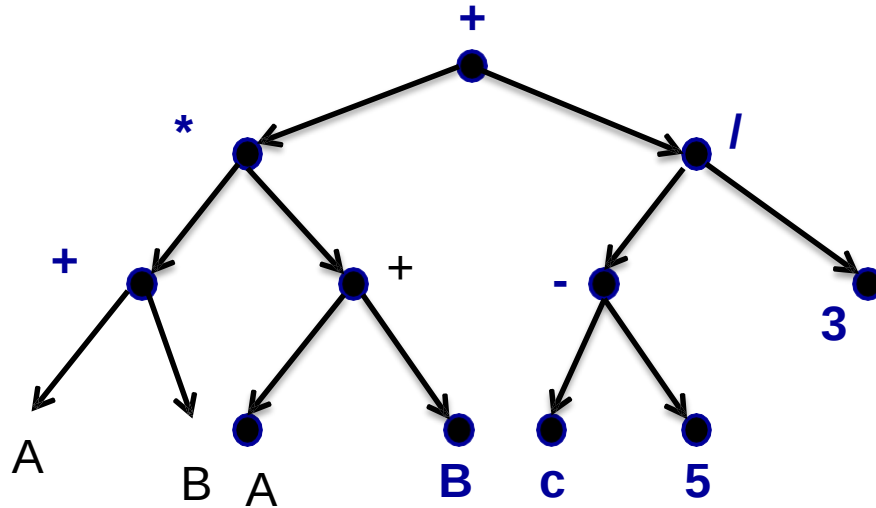
$$2 \times (5 - 1) + (3 \times 2)$$



Example: Create the expression tree of $(A + B)^2 + (C - 5) / 3$

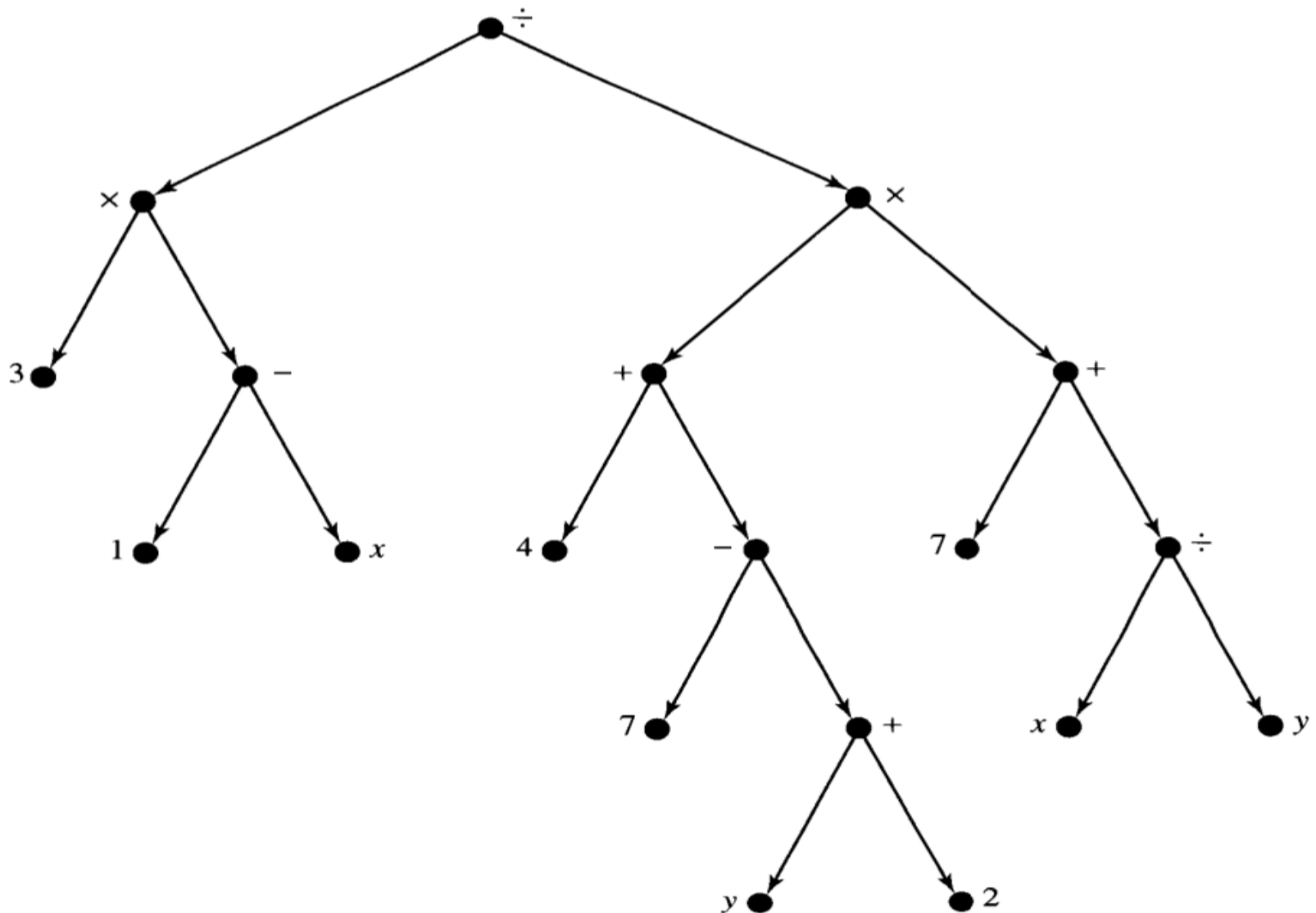
Question

Example: Create the expression tree of $(A + B)^2 + (C - 5) / 3$



Question

$$(3 \times (1 - x)) \div ((4 + (7 - (y + 2))) \times (7 + (x \div y))).$$



Question

In Exercises 1 through 10, construct the tree the algebraic expression.

1. $(7 + (6 - 2)) - (x - (y - 4))$

2. $(x + (y - (x + y))) \times ((3 \div (2 \times 7)) \times 4)$

3. $3 - (x + (6 \times (4 \div (2 - 3))))$

4. $((((2 \times 7) + x) \div y) \div (3 - 11))$

5. $((2 + x) - (2 \times x)) - (x - 2)$

6. $(11 - (11 \times (11 + 11))) + (11 + (11 \times 11))$ —

Infix, Prefix, Postfix Expressions

Infix, Postfix and Prefix Expressions

- ☛ INFIX: the expressions in which operands surround the operator, e.g. $x+y$, $6*3$ etc this way of writing the Expressions is called infix notation.
 - ☛ POSTFIX: Postfix notation are also Known as Reverse Polish Notation (RPN). They are different from the infix and prefix notations in the sense that in the postfix notation, operator comes after the operands, e.g. $xy+$, $xyz+^*$ etc.
 - ☛ PREFIX: Prefix notation also Known as Polish notation. In the prefix notation, operator comes before the operands, e.g. $+xy$, $+xyz$ etc.
-

Infix, Prefix, Postfix Expressions

WHY??

- ☛ Why to use PREFIX and POSTFIX notations when we have simple INFIX notation?
 - ☛ INFIX notations are not as simple as they seem specially while evaluating them. To evaluate an infix expression we need to consider Operators' Priority and Associative property
 - E.g. expression $3+5*4$ evaluate to 32 i.e. $(3+5)*4$ or to 23 i.e. $3+(5*4)$.
 - ☛ To solve this problem Precedence or Priority of the operators were defined. Operator precedence governs evaluation order. An operator with higher precedence is applied before an operator with lower precedence.
-

Infix, Prefix, Postfix Expressions

Infix,
[Operand] **operator** [operand]

Postfix,
[Operand] [operand] **operator**

Prefix,
operator [Operand] [operand]

Examples of infix to prefix and post fix

Infix	Postfix	Prefix
A+B	AB+	+AB
(A+B) * (C + D)	AB+CD+*	*+AB+CD
A-B/(C*D^E)	ABCDE^*/-	-A/B*C^DE

Infix, Prefix, Postfix Expressions

Suppose that we would like to rewrite
 $A+B*C$ in postfix

☛ Applying the rules of precedence, we obtained

$A+B*C$

$A+(B*C)$ Parentheses for emphasis

$A+(BC^*)$ Convert the multiplication, Let $D=BC^*$

$A+D$ Convert the addition

$A(D)+$

$ABC^*+ \quad \text{Postfix Form}$

Infix, Prefix, Postfix Expressions

Postfix Examples

Infix	Postfix	Evaluation
$2 - 3 * 4 + 5$	$2\ 3\ 4\ *\ -\ 5\ +$	-5
$(2 - 3) * (4 + 5)$	$2\ 3\ -\ 4\ 5\ +\ *$	-9
$2 - (3 * 4 + 5)$	$2\ 3\ 4\ *\ 5\ +\ -$	-15

☞ Why ? No brackets necessary !

Infix, Prefix, Postfix Expressions

For, example consider the following arithmetic Infix to Prefix expression

$$((A+B) * (C+D) / (E-F)) + G$$

Prefix expression is

$$+ / * + A B + C D - E F G$$

Infix, Prefix, Postfix Expressions

Prefix, Infix, Postfix Notation

Infix --> Prefix

$((A+B) * C - D) * E$

$(+AB * C - D) * E$

$(*+ABC - D) * E$

$- * + ABCD * E$

$* - * + ABCDE$

Infix --> Postfix

$((A+B) * C - D) * E$

$(AB+ * C - D) * E$

$(AB+C* - D) * E$

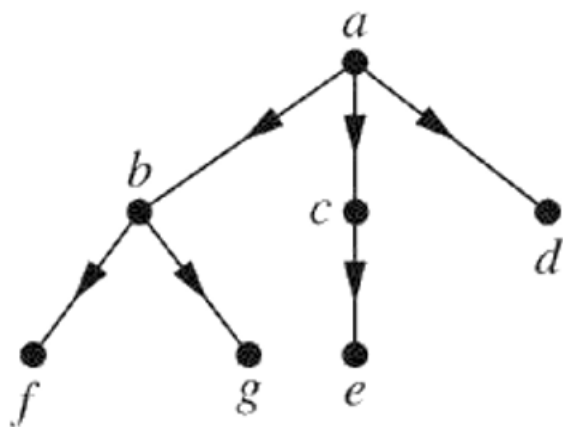
$AB+C*D- * E$

$AB+C*D-E*$

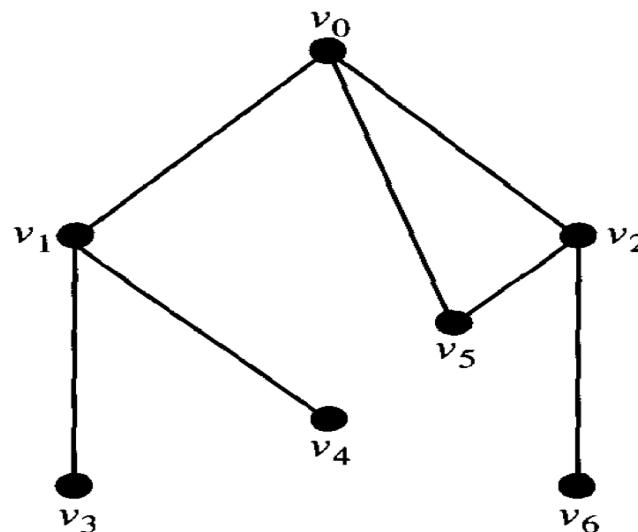
Undirected Trees

Let R be a symmetric relation on a set A . Then the following statements are equivalent.

- (a) R is an undirected tree
- (b) R is connected and acyclic. (no loops as loop is a cycle)



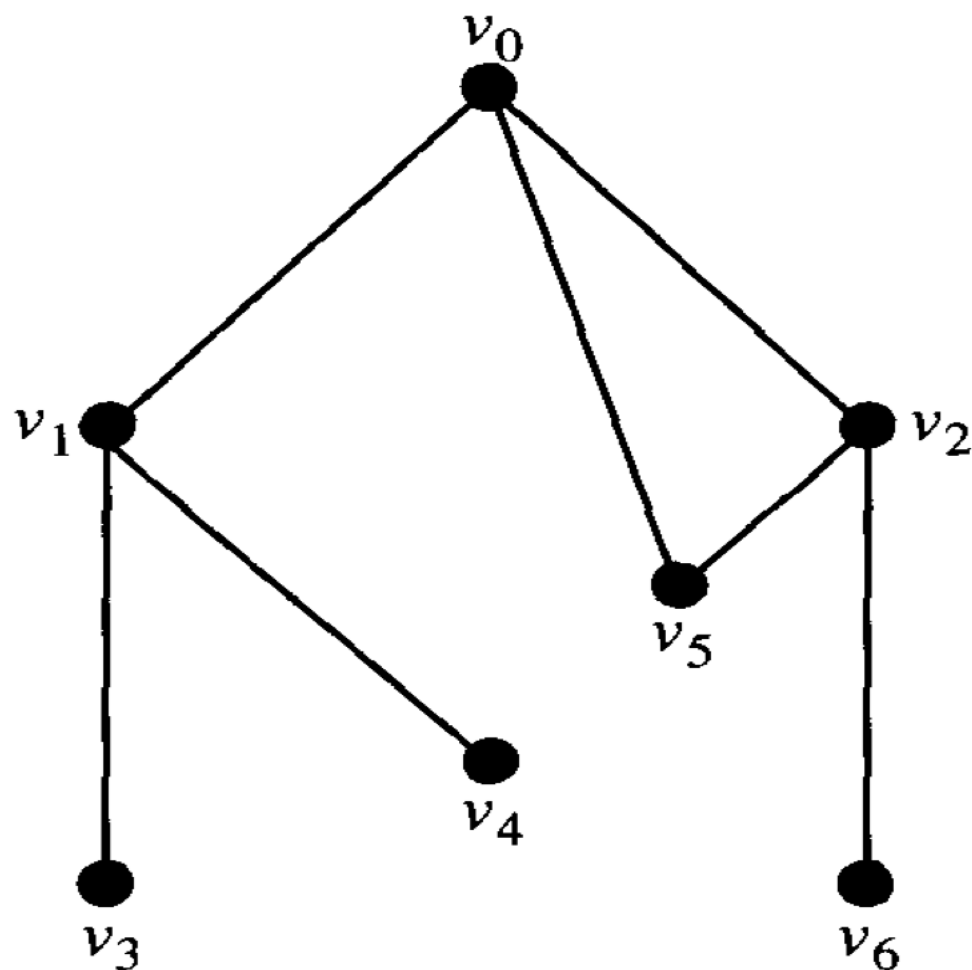
Directed tree



Undirected tree

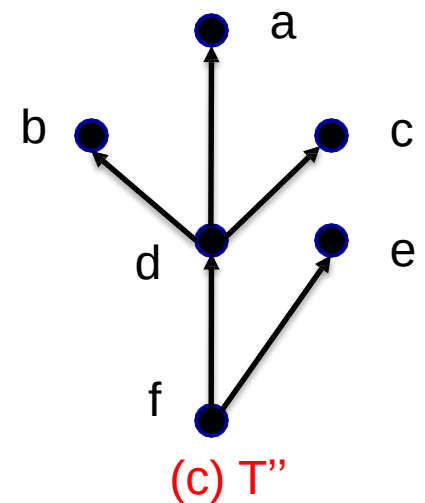
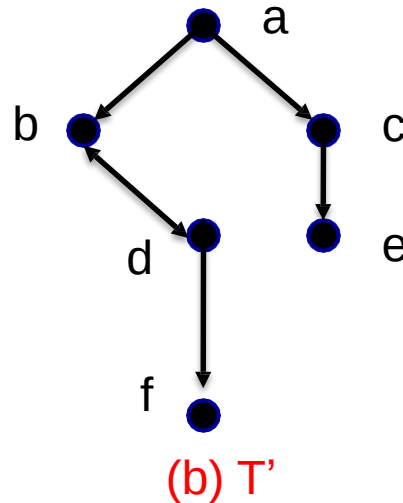
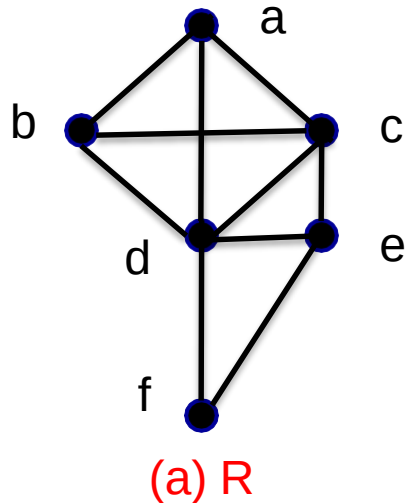
Undirected tree

	v_0	v_1	v_2	v_3	v_4	v_5	v_6
v_0	0	1	1	0	0	1	0
v_1	1	0	0	1	1	0	0
v_2	1	0	0	0	0	1	1
v_3	0	1	0	0	0	0	0
v_4	0	1	0	0	0	0	0
v_5	1	0	1	0	0	0	0
v_6	0	0	1	0	0	0	0



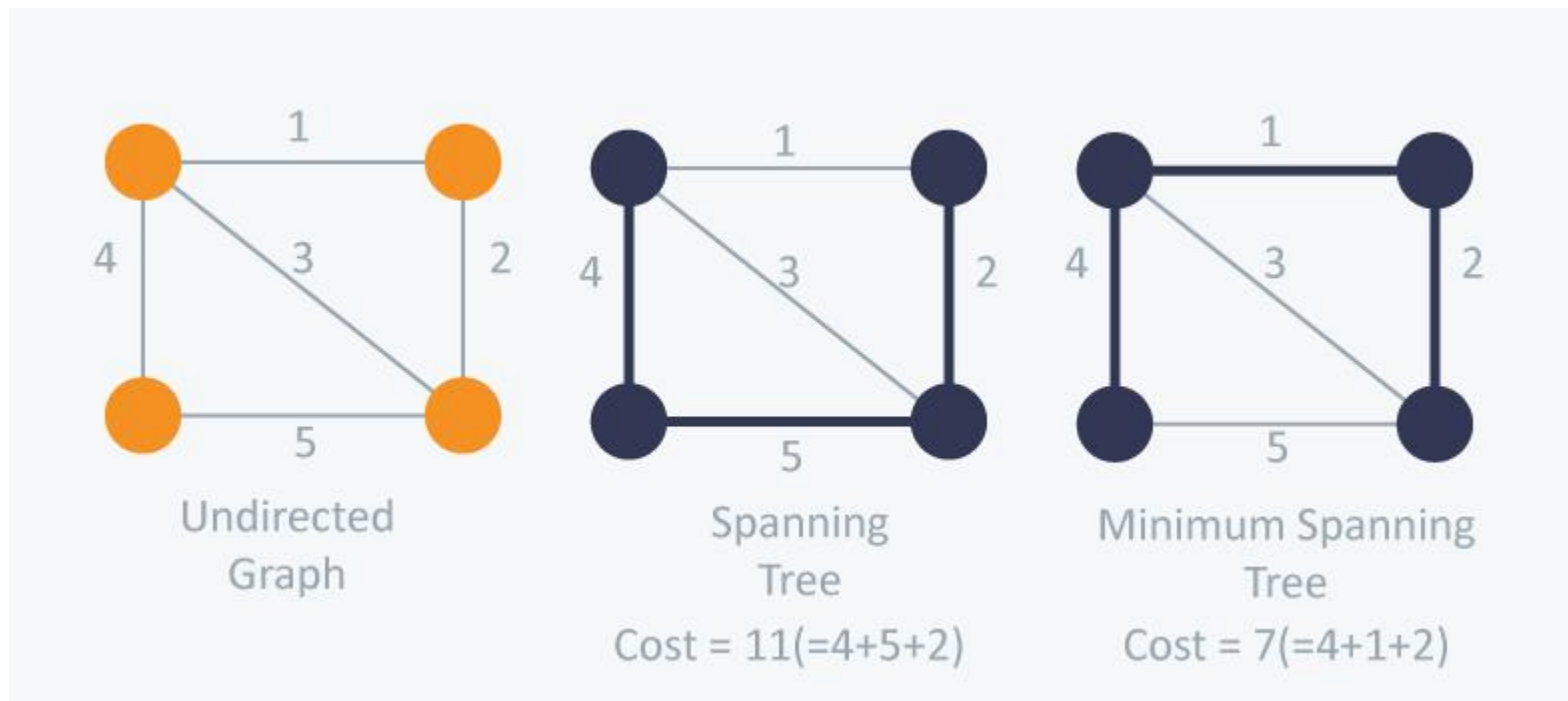
Spanning Tree

If R is a symmetric, connected relation on a set A , we say that a tree T on A is a spanning tree for R if T is a tree with exactly the same vertices as R and which can be obtained from R by deleting some edges of R . For instance,



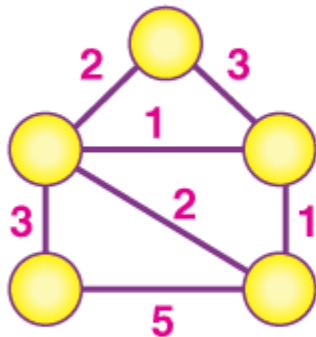
Spanning Tree

If R is a symmetric, connected relation on a set A , we say that a tree T on A is a spanning tree for R if T is a tree with exactly the same vertices as R and which can be obtained from R by deleting some edges of R . For instance,

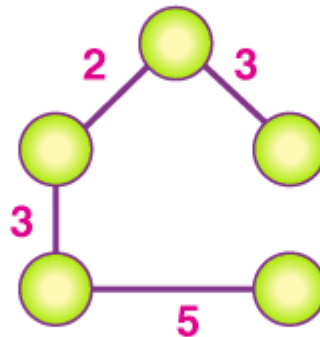


Spanning Tree

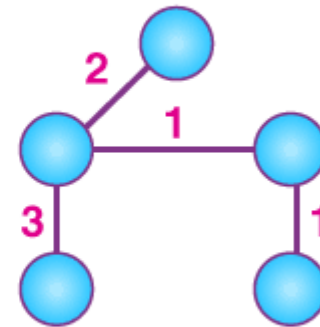
If R is a symmetric, connected relation on a set A , we say that a tree T on A is a spanning tree for R if T is a tree with exactly the same vertices as R and which can be obtained from R by deleting some edges of R . For instance,



Graph



Spanning tree
cost = 13

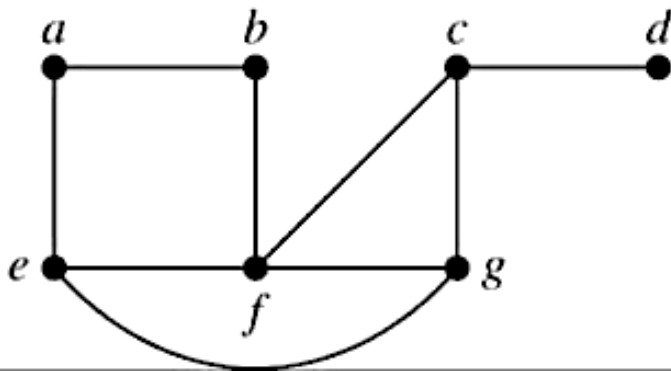


Minimum Spanning
tree, cost = 7

Spanning Trees

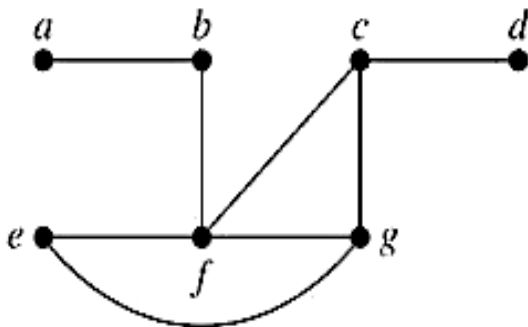
Let G be a simple graph. A **spanning tree** of G is a subgraph of G that is a tree containing every vertex of G .

Example 1 Find a spanning tree of G .



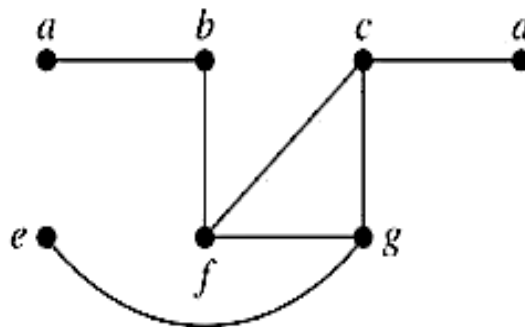
Sol.

Remove an edge from any circuit.
(repeat until no circuit exists)



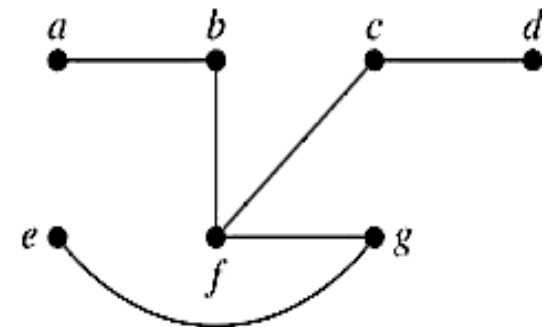
Edge removed: $\{a, e\}$

(a)



$\{e, f\}$

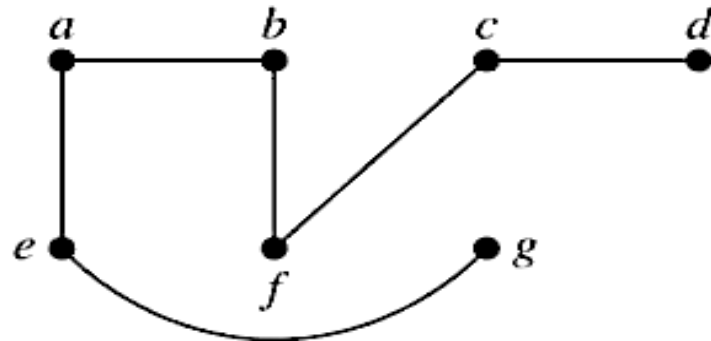
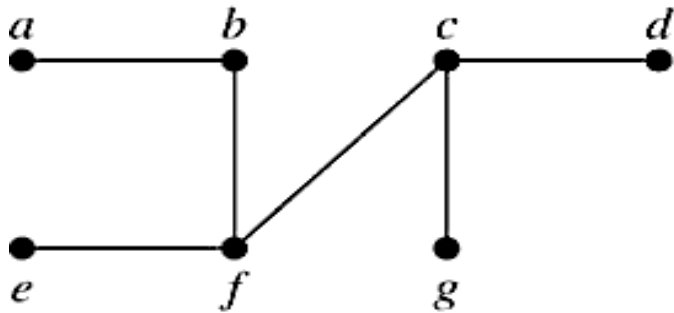
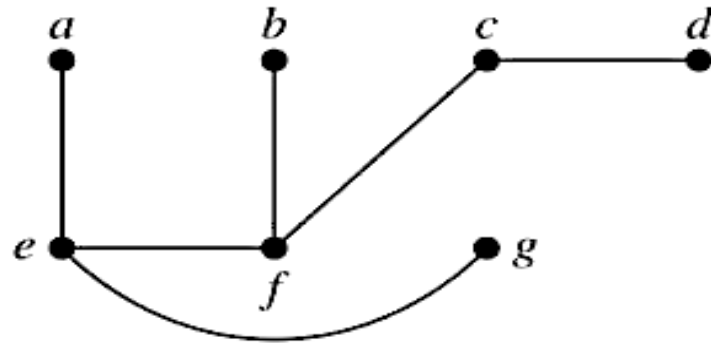
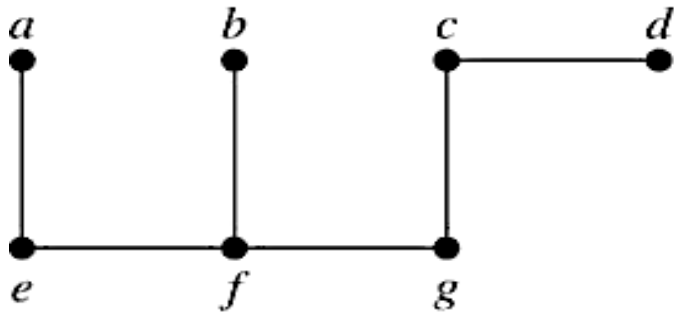
(b)



$\{c, g\}$

(c)

Four spanning trees of G :

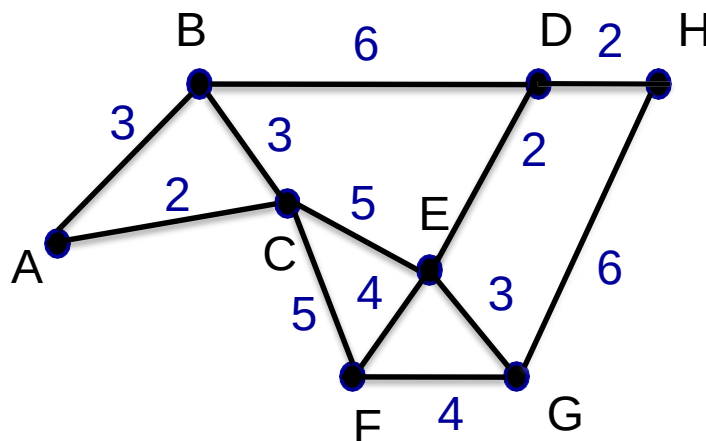


Note: A simple graph is connected if and only if it has a spanning tree.

Minimal Spanning Trees

- **Weighted graph**

A weighted graph is a graph for which each edges is labeled with a numerical value called its weight.

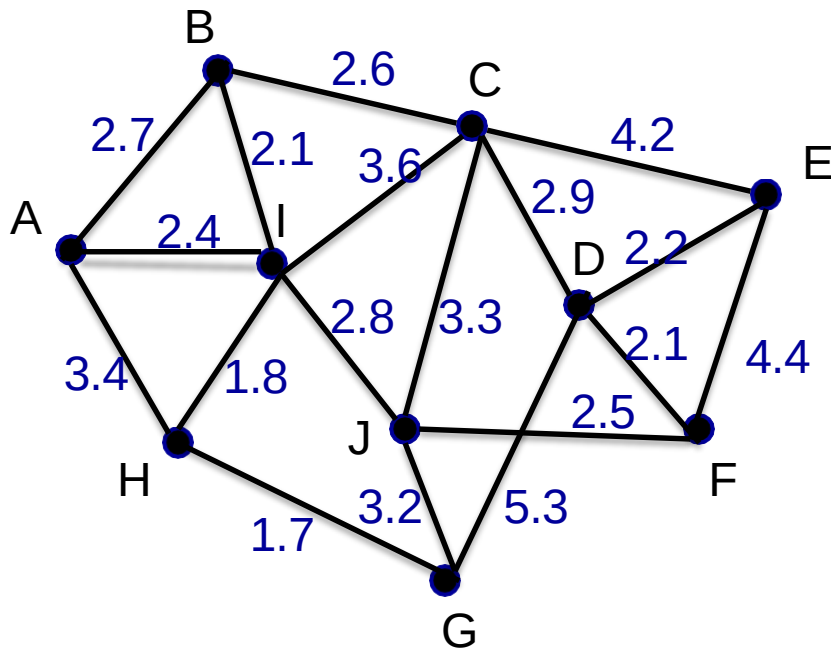


vertices: towns

edges: the distance between two connected towns

Minimal Spanning Trees

■ Example 2



Vertices: relay stations

Edges: the cost for upgrading

Minimal Spanning Trees

- Distance between vertices

The weight of an edge (v_i, v_j) is sometimes referred to as the distance between vertices v_i and v_j .

- Nearest neighbor

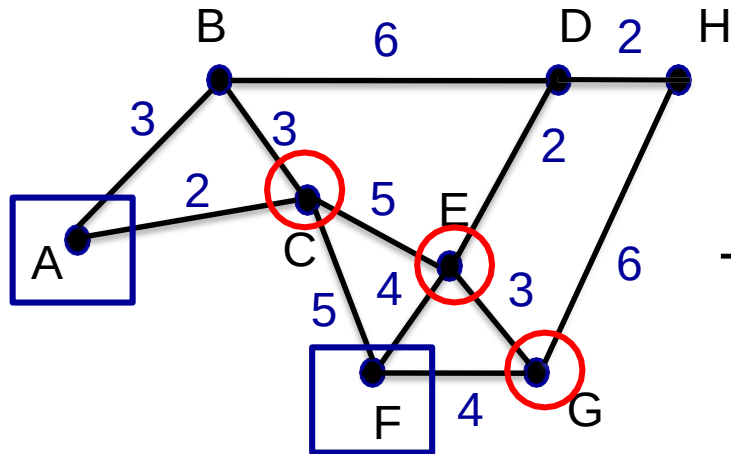
A vertex u is nearest neighbor of vertex v

if u and v are adjacent and no other vertex is joined to v by an edge of lesser weight than (u, v)

(Note: may have more than one nearest neighbor)

Minimal Spanning Trees

■ Example 3



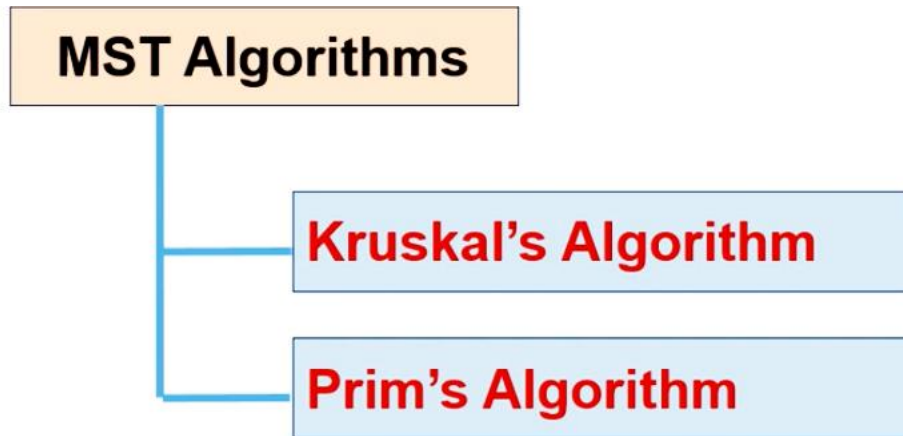
The nearest neighbor of A is C

The nearest neighbor of F is E & G

Minimal Spanning Trees

An undirected spanning tree in which the total weight of the edges is as small as possible, is called a minimal spanning tree.

- **Two greedy algorithms** for minimum spanning tree:



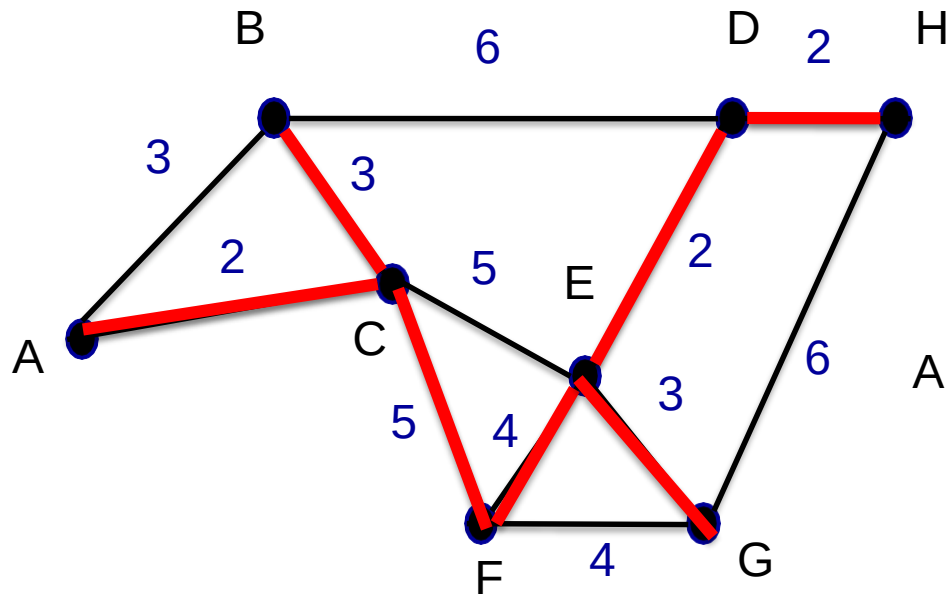
7.5 Minimal Spanning Trees

Algorithm: Prim's ALGORITHM

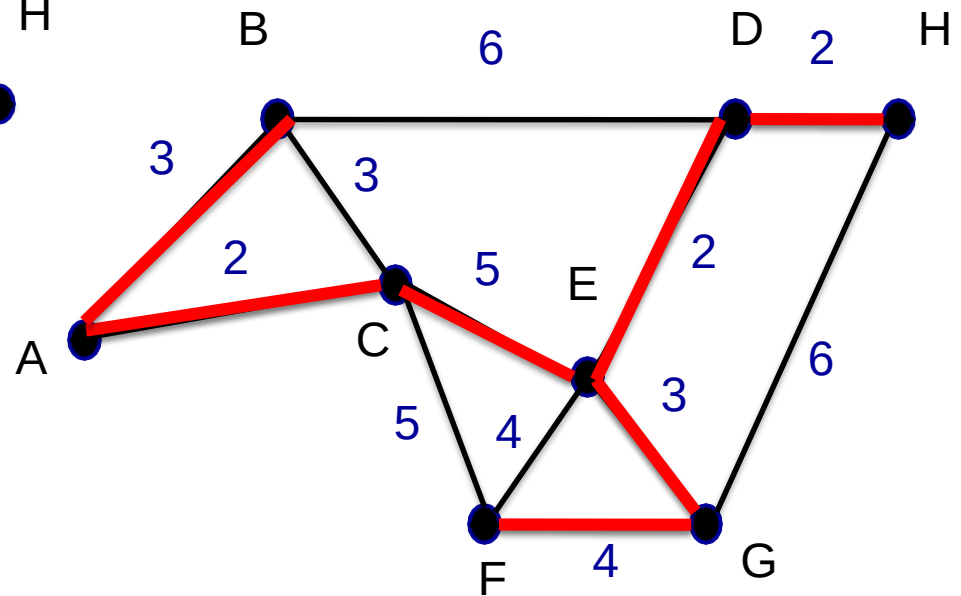
- **Step 1:** Choose a vertex v_1 of R . Let $V=\{v_1\}$ and $E=\{\}$;
- **Step 2:** Choose a nearest neighbor v_i of V that is adjacent to v_j , v_j in V , and for which the edge (v_i, v_j) **does not form a cycle with members of E** . Add v_i to V and add (v_i, v_j) to E .
- **Step 3:** Repeat Step 2 until $|E|=n-1$. Then V contains all n vertices of R , and E contains the edges of a minimal spanning tree for R .
- معروف انها ستحتوي علي عدد n فيرتسيوز و عدد $n-1$ ايدجيس

7.5 Minimal Spanning Trees

■ Example 1



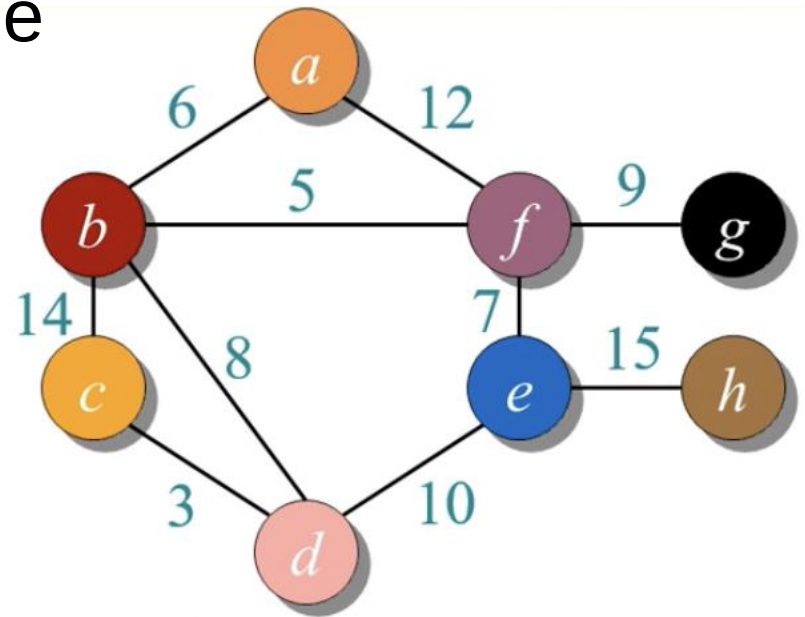
21 Kilometers



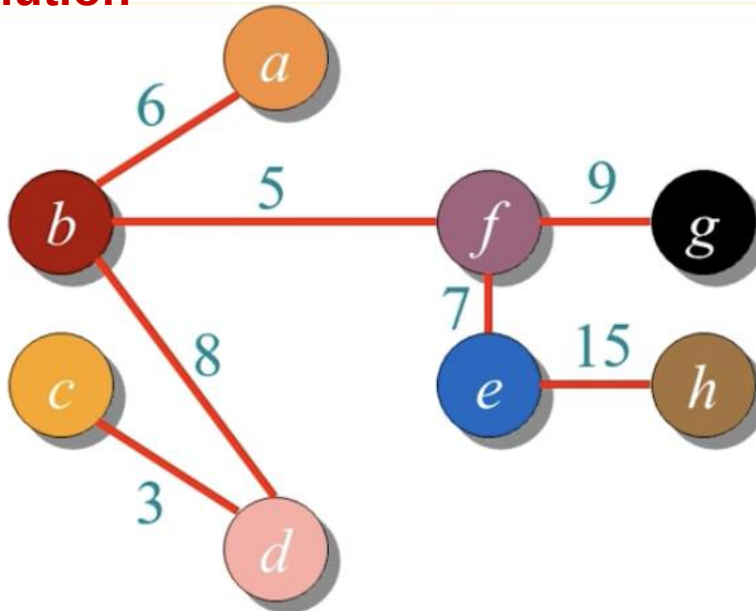
21 Kilometers

Question

Using Prim algorithm, find the MST starting from vertex c.

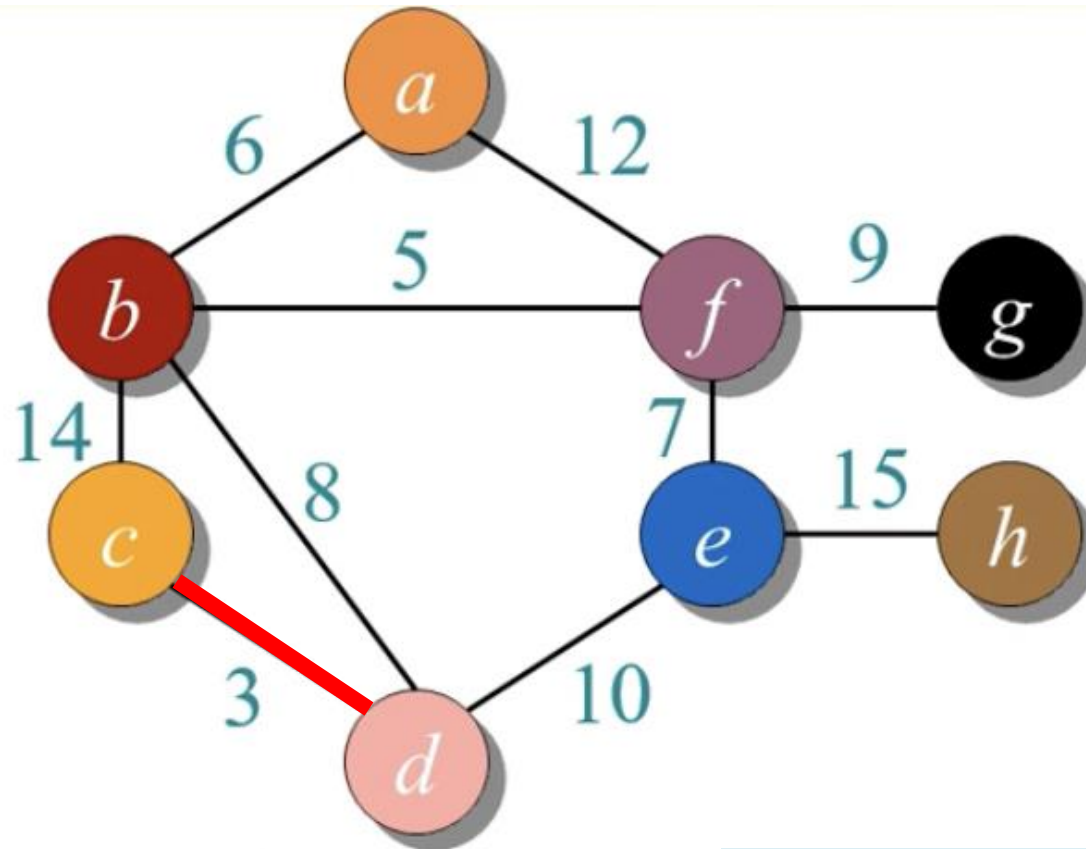


solution



cdbfaegh

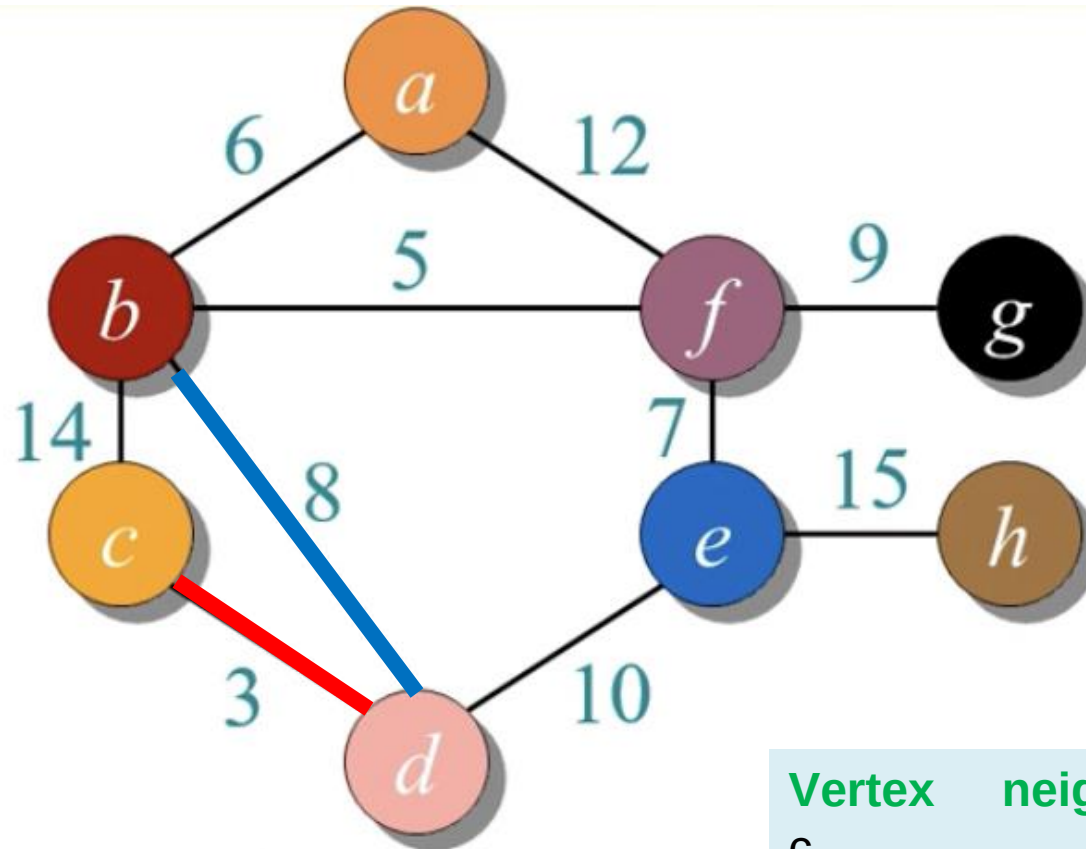
Iter 1



Select d

Vertex	neighbors
c	b, d

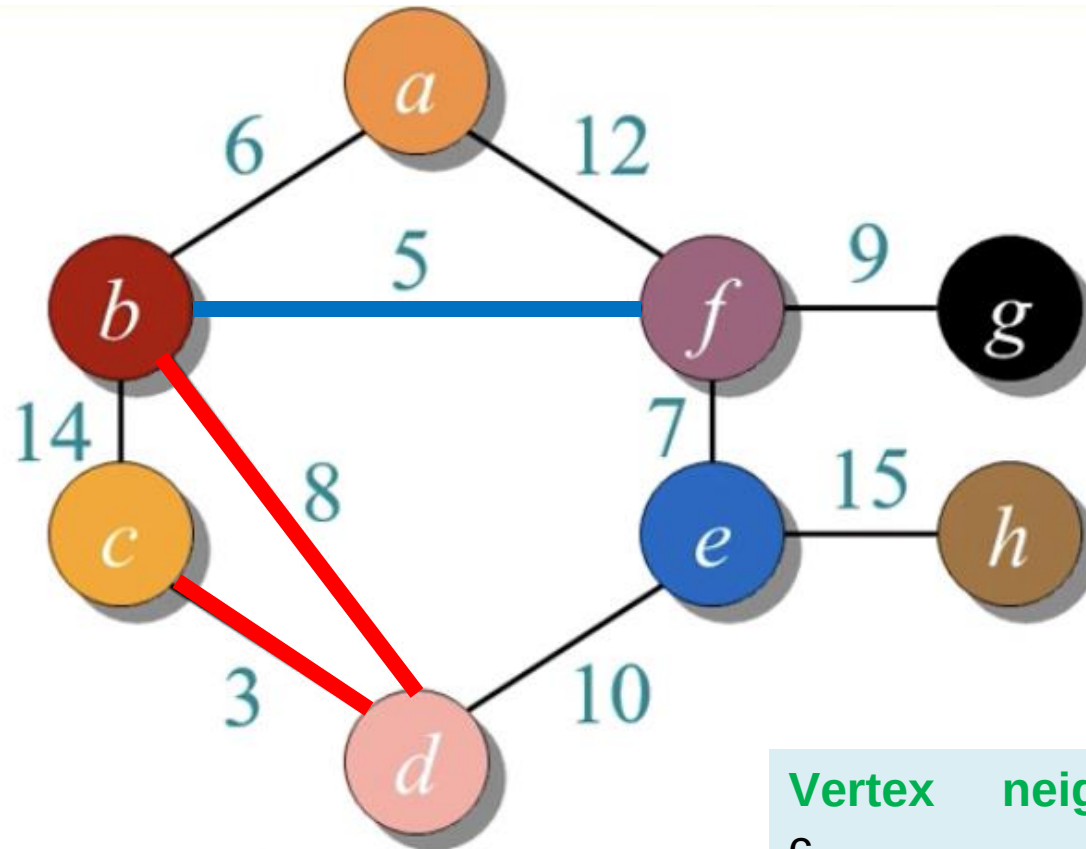
Iter 2



Vertex	neighbors
c	-
d	e

Select b

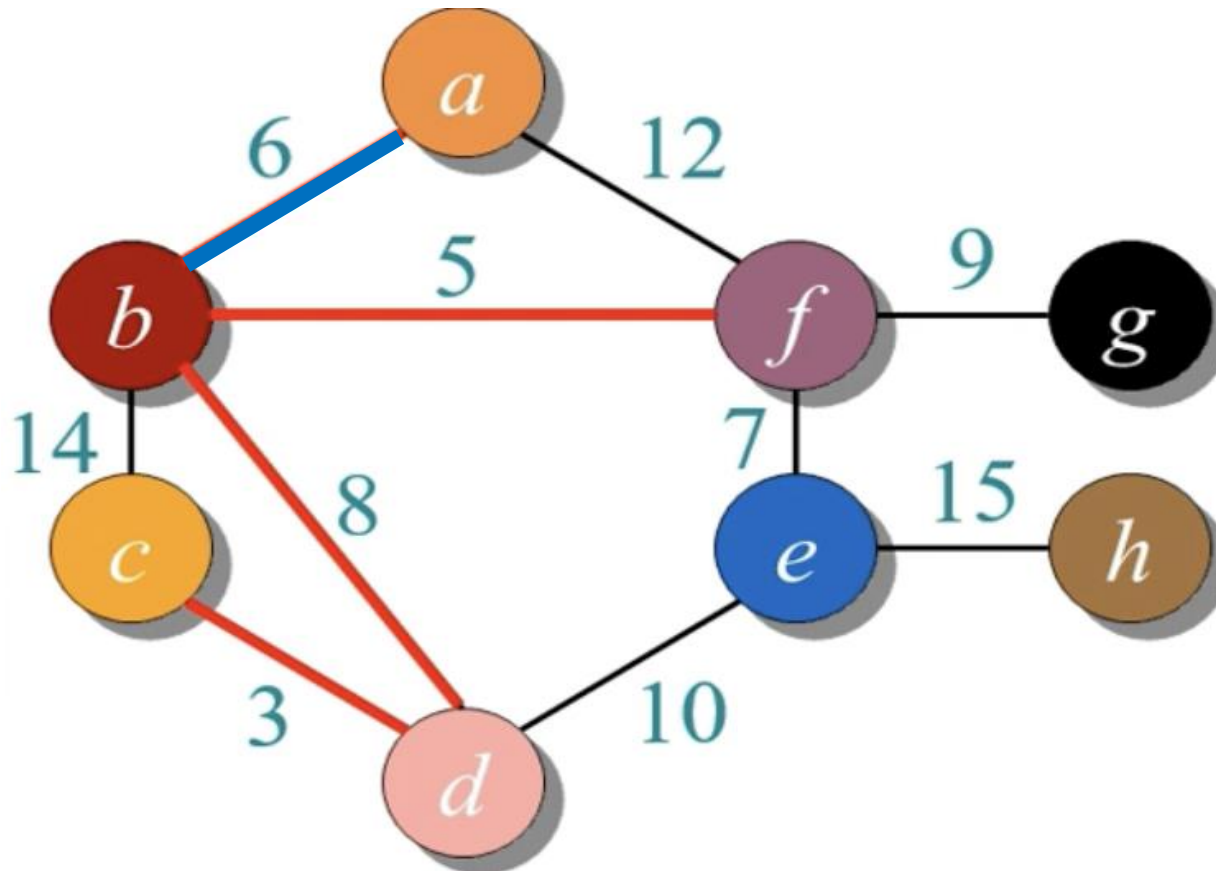
Iter 3



Select f

Vertex	neighbors
c	-
d	e
b	a,f

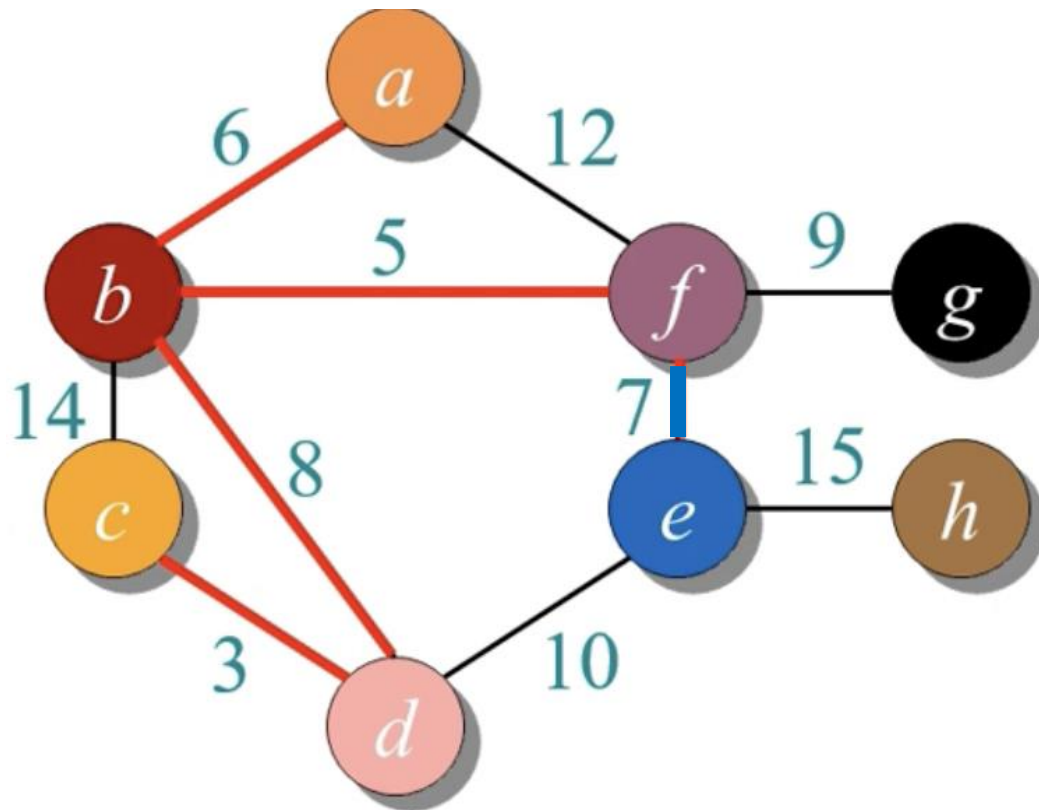
iter4



Vertex	neighbors
c	-
d	e
b	a
f	a, e, g

Select a

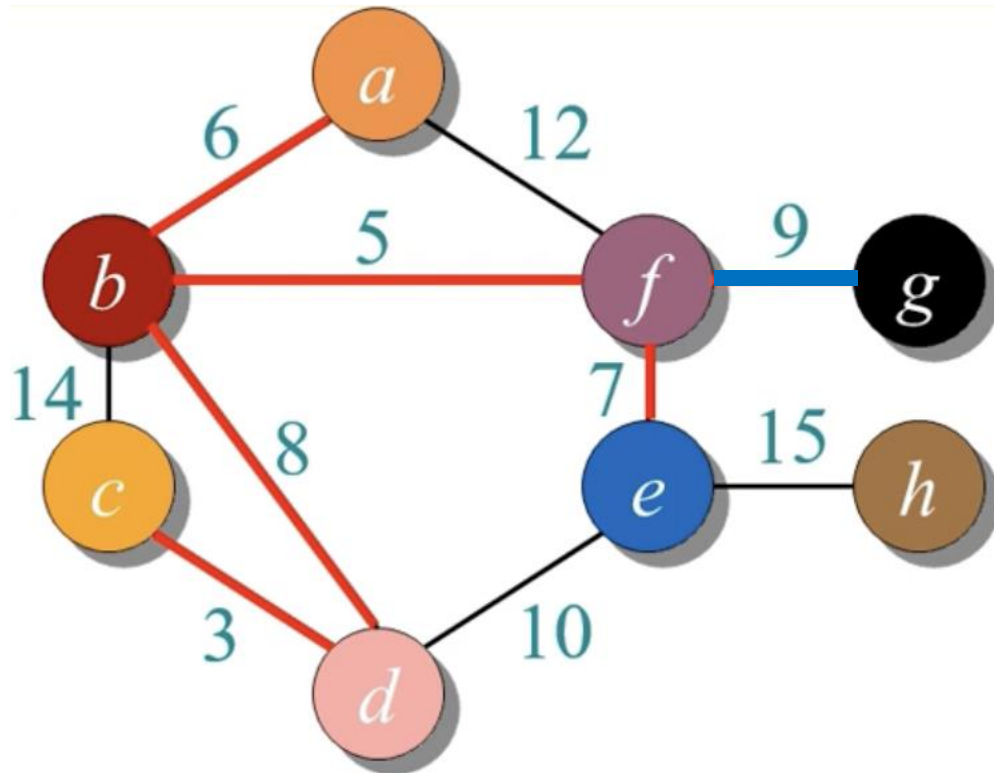
Iter 5



Vertex	neighbors
c	-
d	e
b	-
f	e, g

Select e

Iter 6



Vertex	neighbors
--------	-----------

c	-
---	---

d	-
---	---

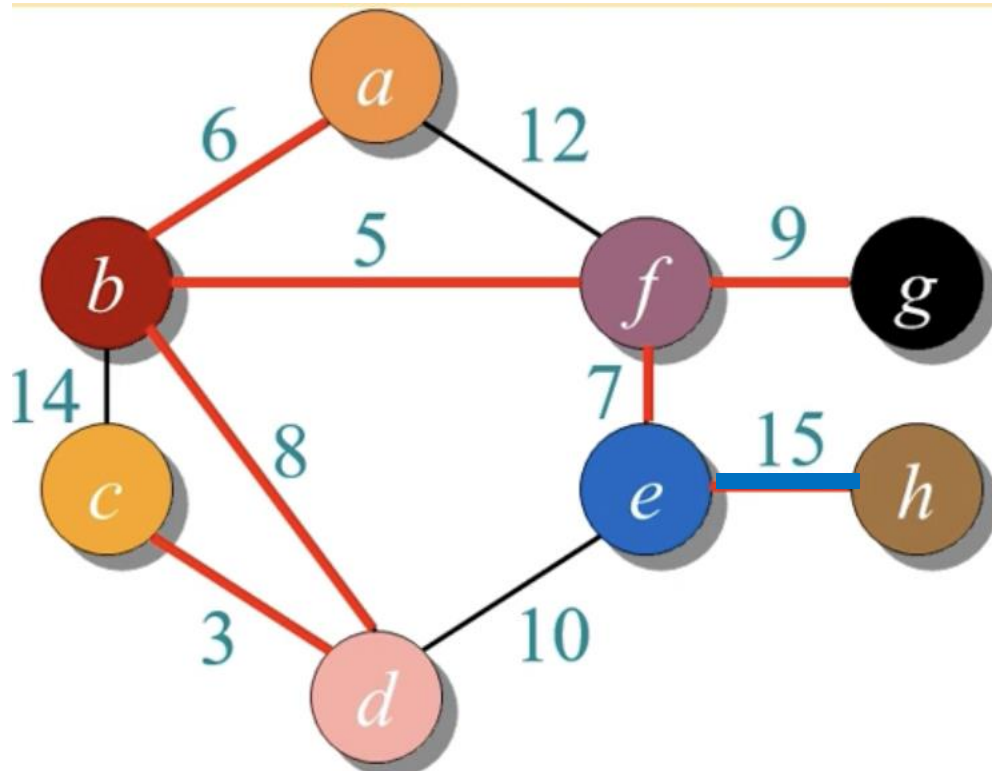
b	-
---	---

f	g
---	---

e	h
---	---

Select g

Iter 7



Vertex	neighbors
--------	-----------

c	-
---	---

d	-
---	---

b	-
---	---

f	-
---	---

e	h
---	---

Select h

$$\text{Cost} = 3 + 8 + 5 + 6 + 7 + 9 + 15 = 53$$

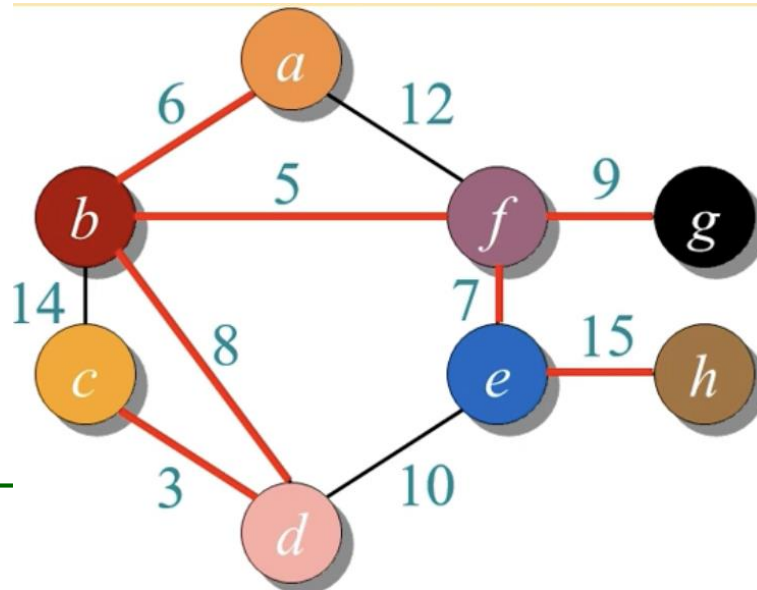
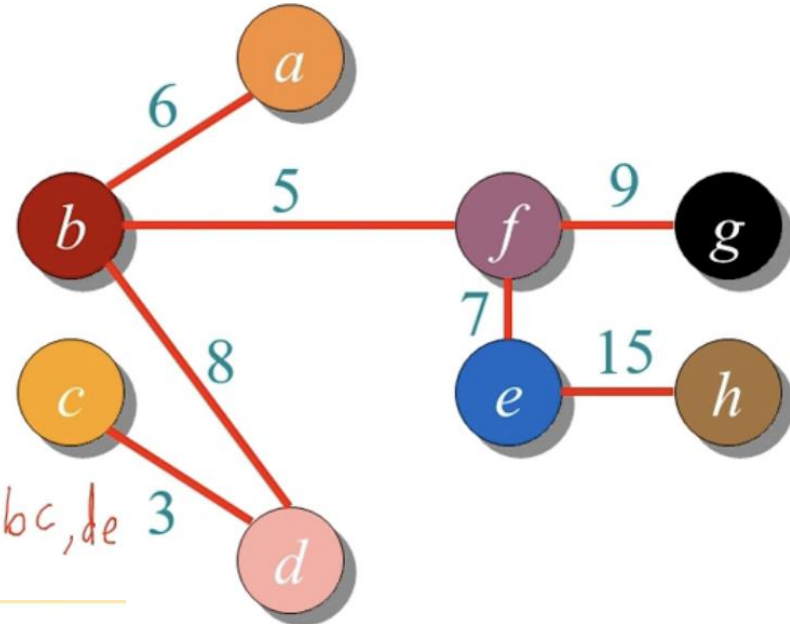
Answer the following

1- Which vertex will be selected at iteration 2? *b*

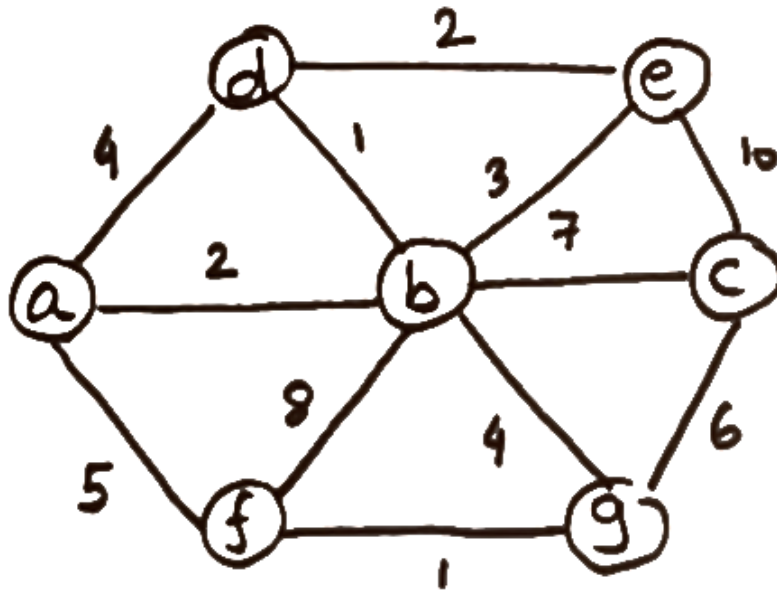
2- Which vertex will be selected at iteration 4? *a*

3- Which vertex will be selected at iteration 7? *h*

4- State an edge which is not selected in MST. *af, bc, de*

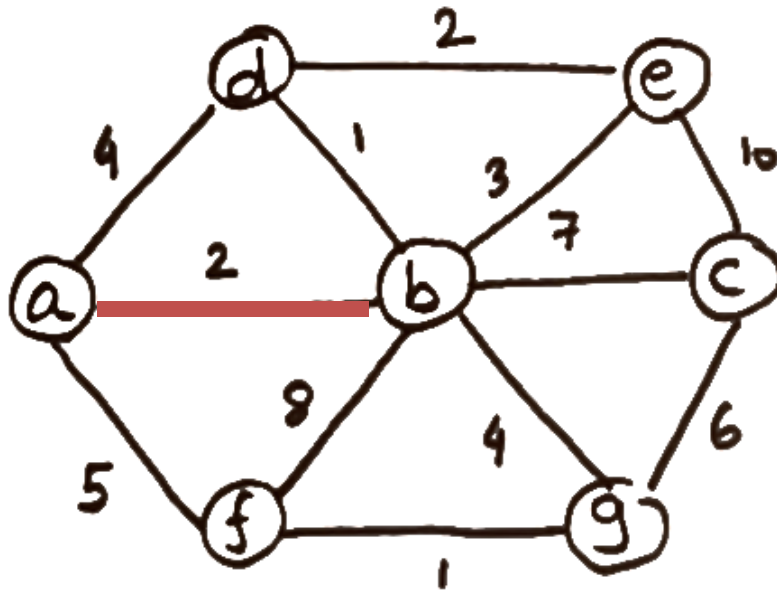


Example



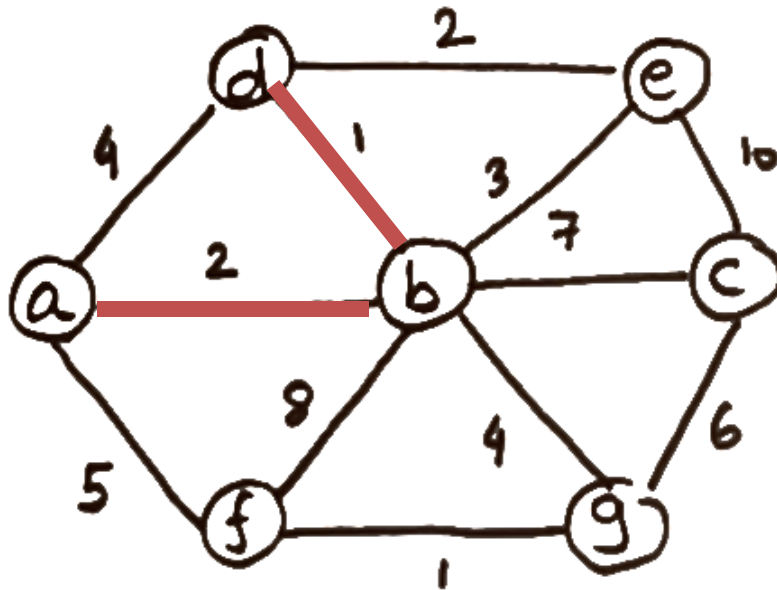
Find MST starting from a

Iter 1



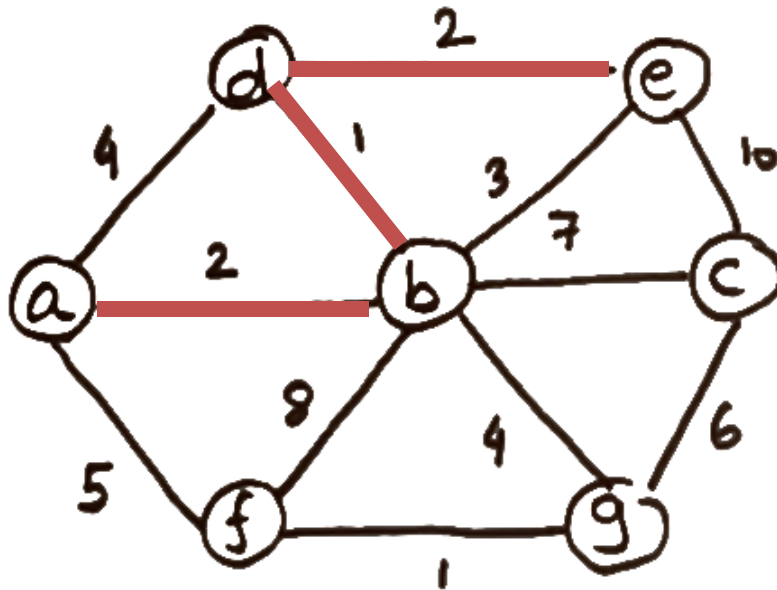
Select b

Iter 2



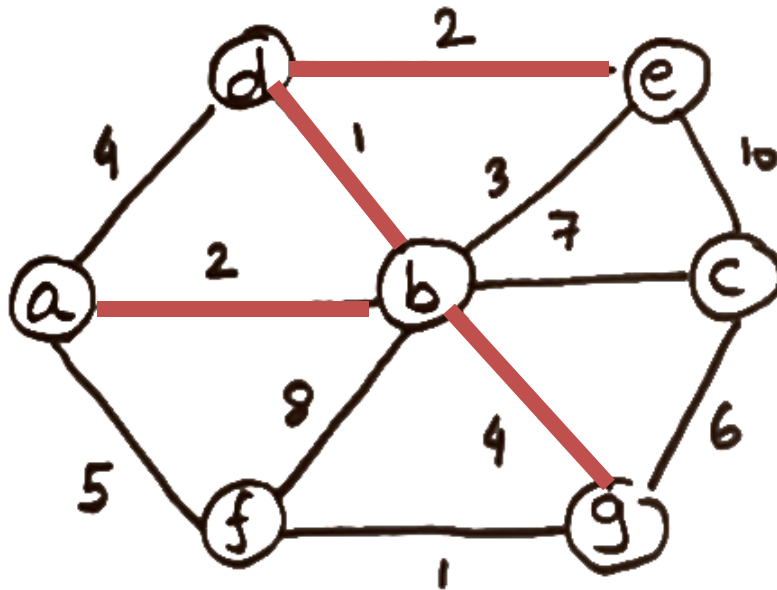
Select d

Iter 3



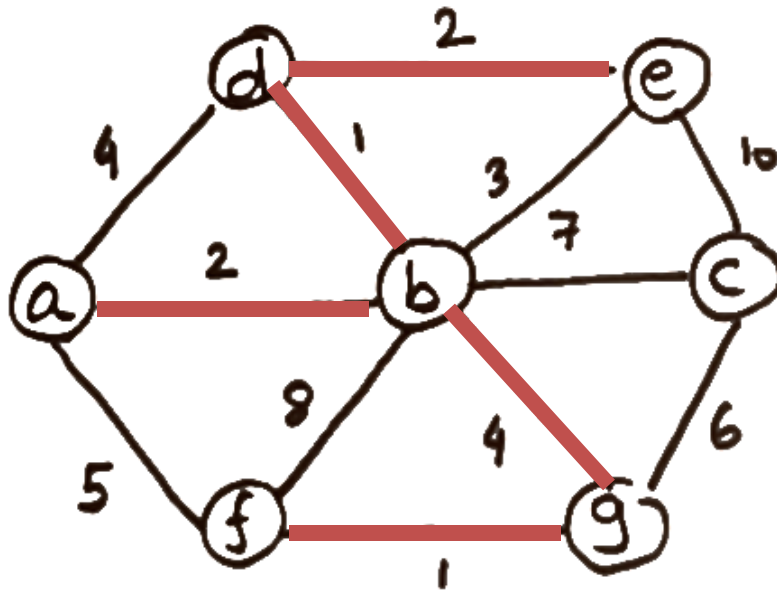
Select e

Iter 4



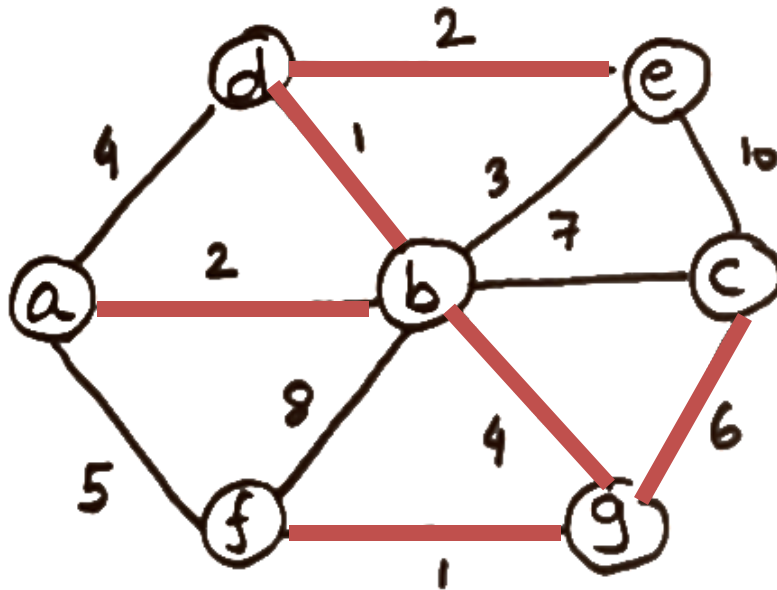
Select g

Iter 5



Select f

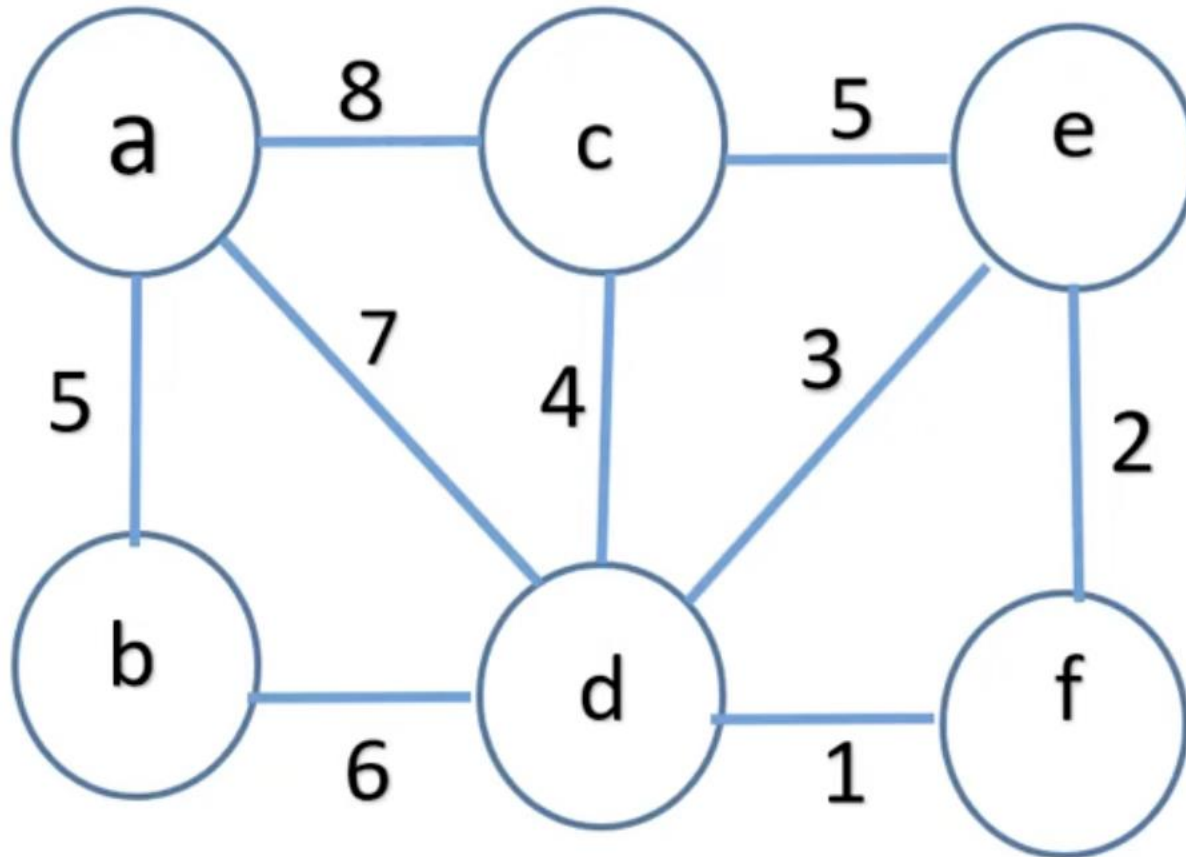
Iter 6



$$\text{Cost} = 1 + 2 + 2 + 4 + 1 + 6 = 16$$

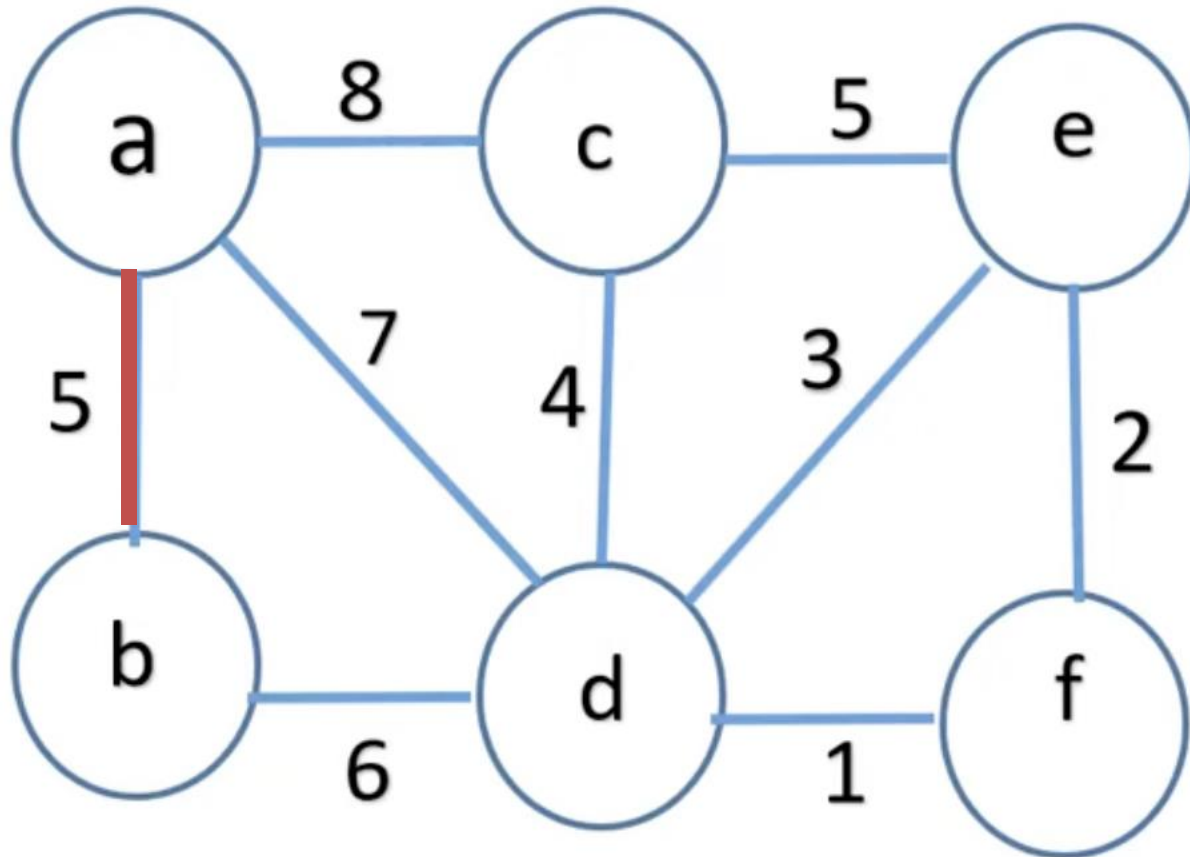
Select c

Question



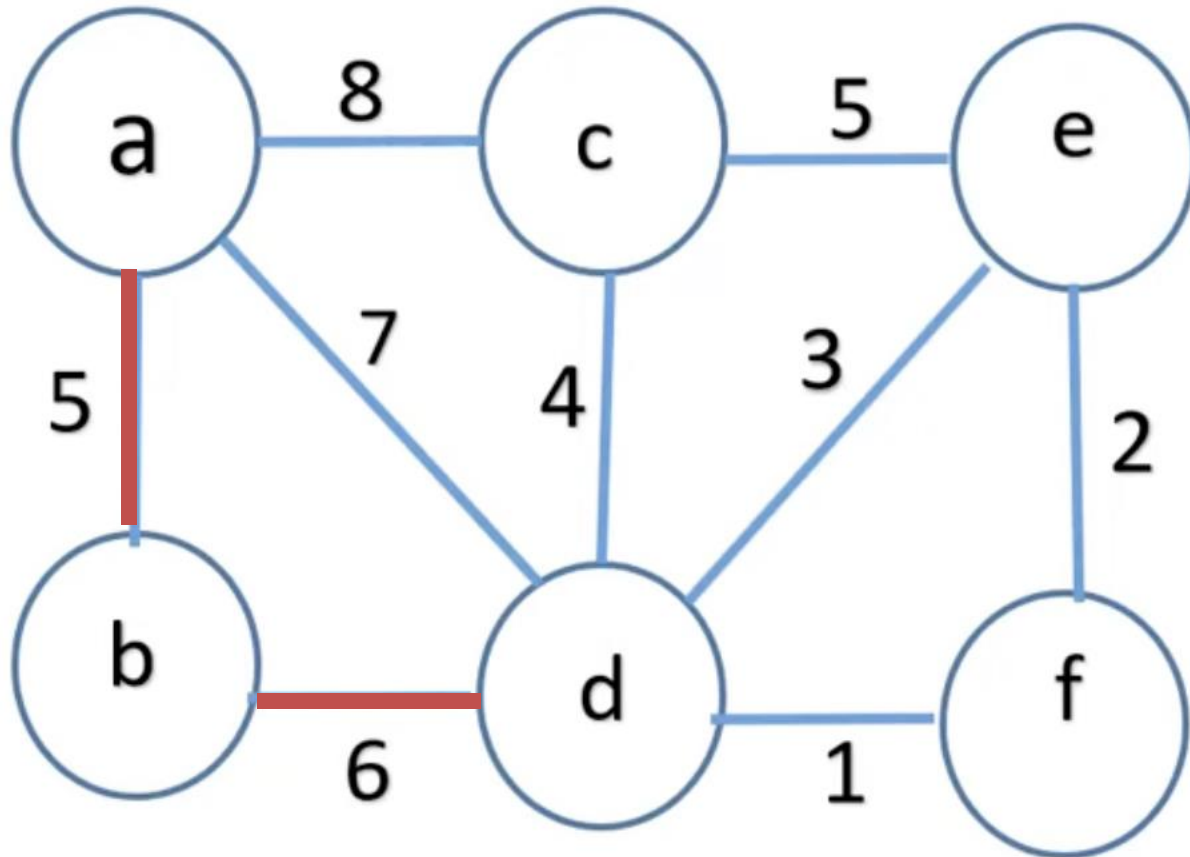
Using Prim algorithm, find the MST starting from vertex a.

Iter 1



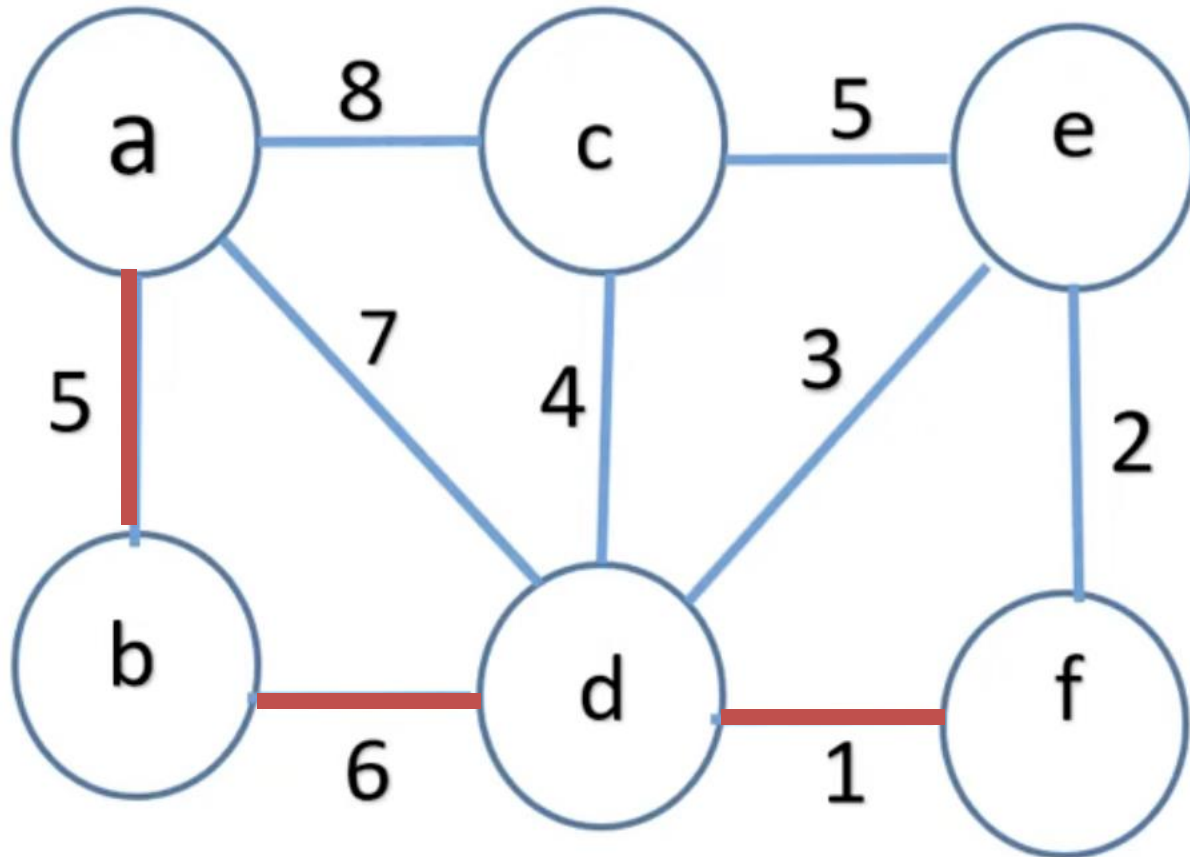
Select b

Iter 2



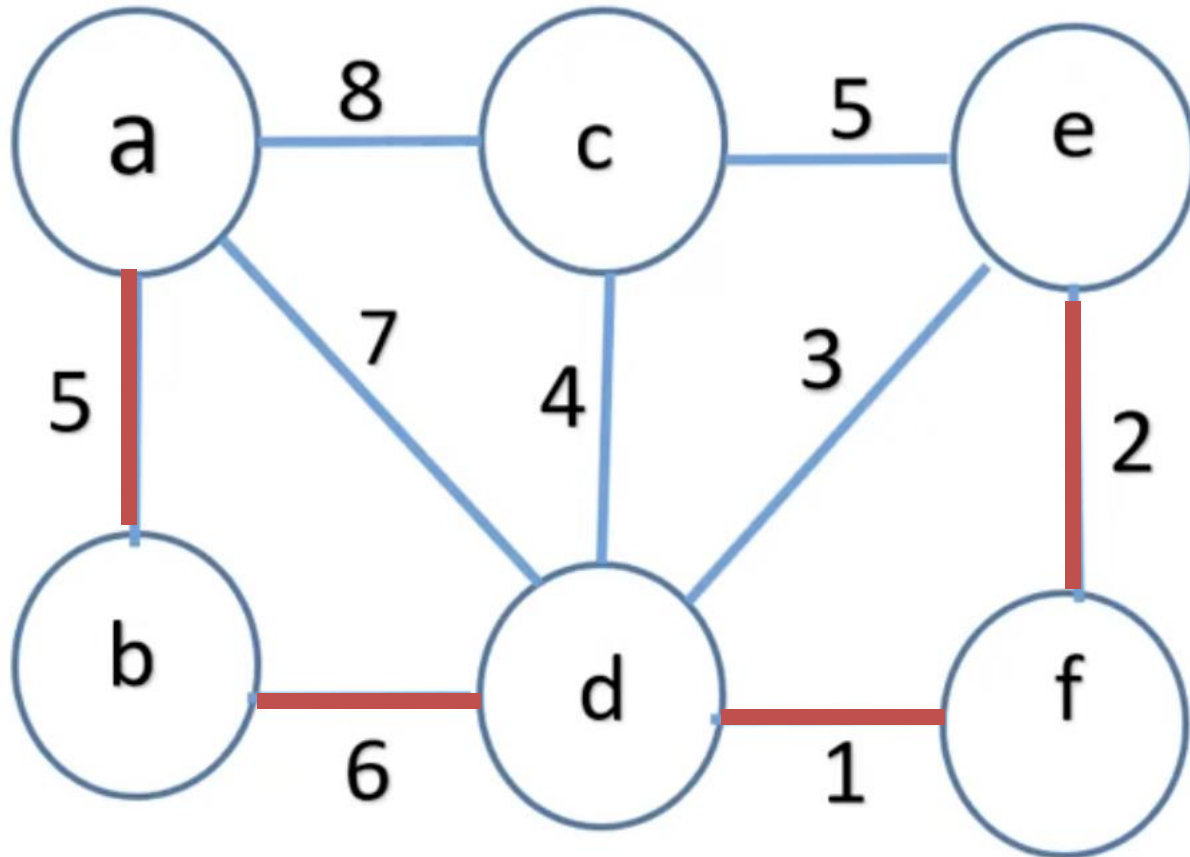
Select d

Iter 3



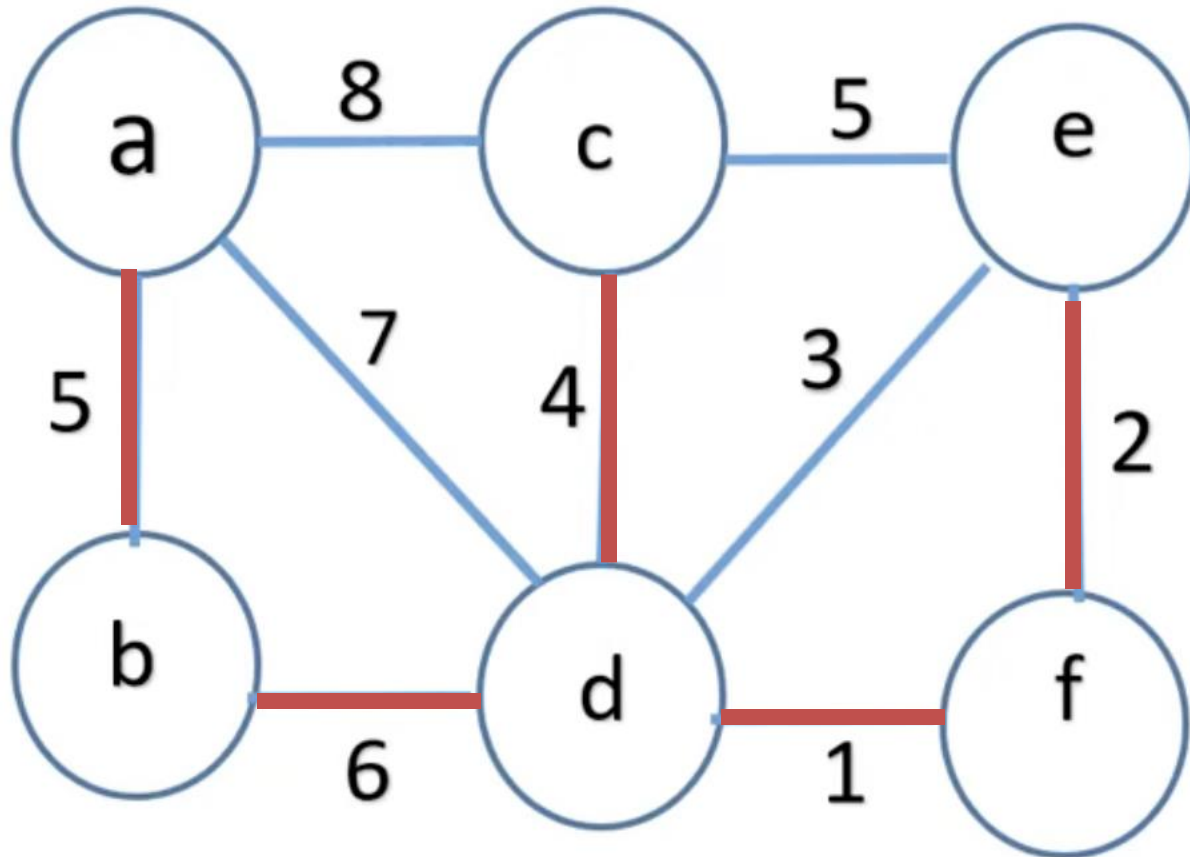
Select f

Iter 4



Select e

Iter 5



Select c

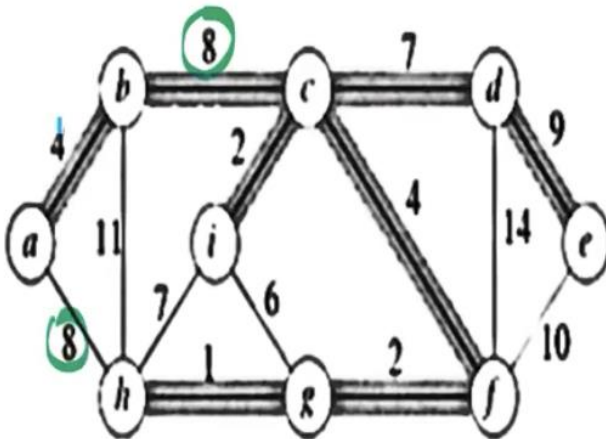
$$\text{Cost} = 5 + 6 + 1 + 2 + 4 = 18$$

Algorithm KRUSKAL'S ALGORITHM

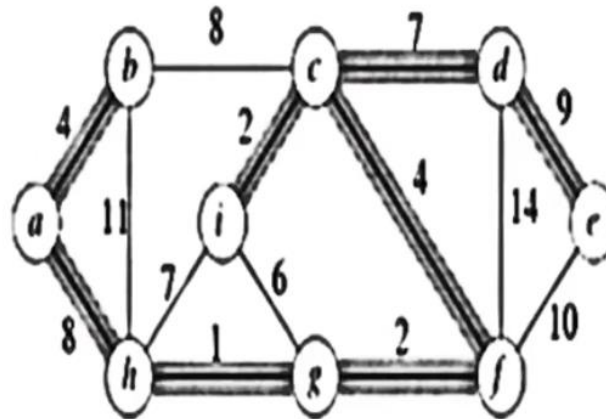
1. Sort the graph edges with respect to their weights.
2. Start adding edges to the MST from the edge with the smallest weight until the edge of the largest weight.
3. Only add edges which doesn't form a cycle, edges which connect only disconnected components.
4. Repeat Step 2 until $|E|=n-1$.

Kruskal algorithm

Adds edge one by one into a growing spanning tree.



Cost = 37

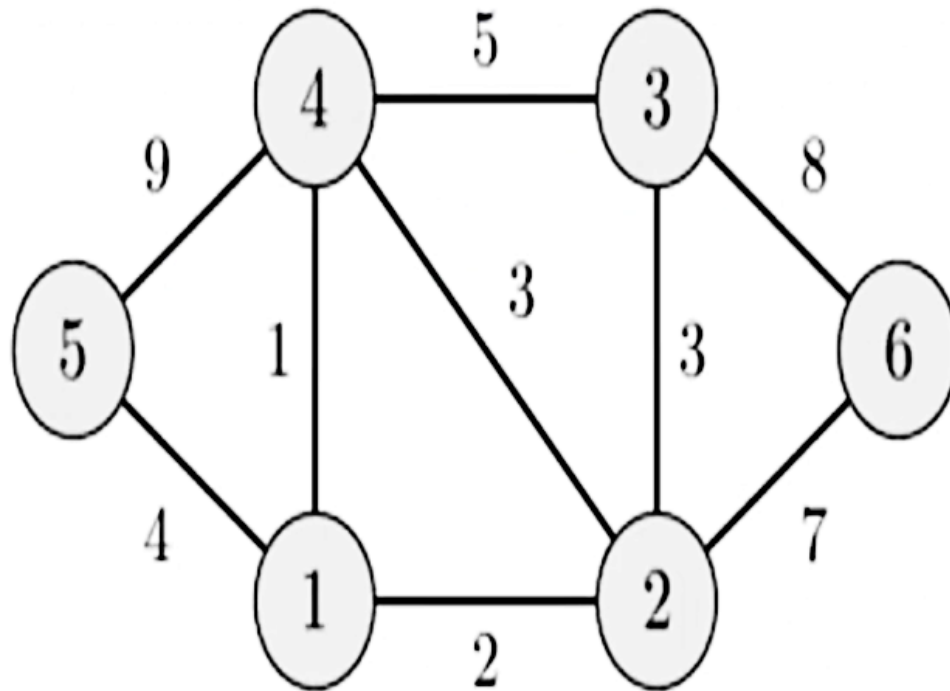


Cost = 37

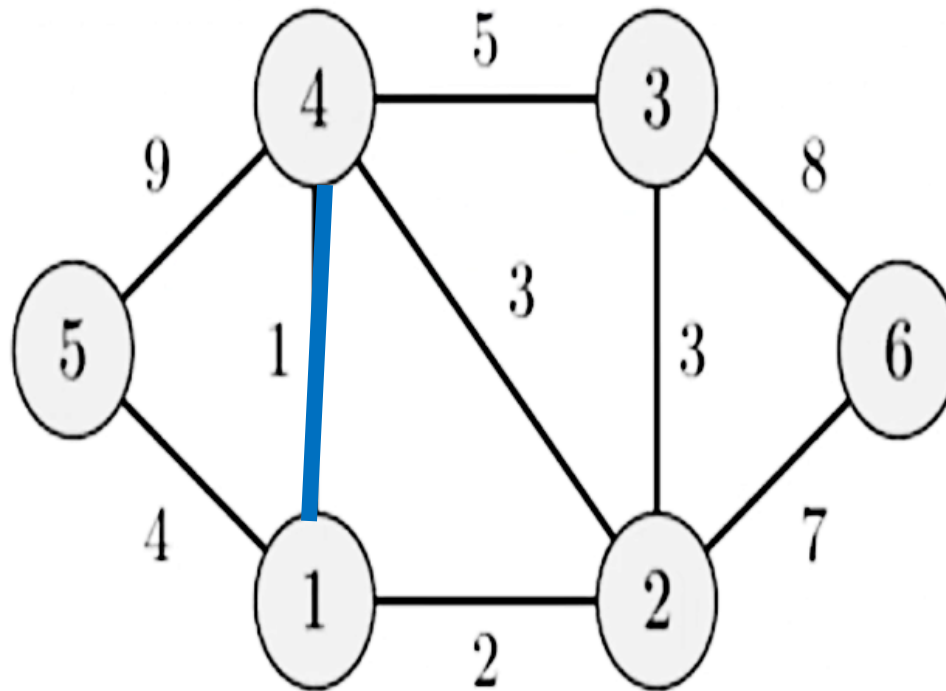
Note: Some Edges have the same cost (Costs : 2,4,7, and 8)

We can obtain more than MST with the same cost if we have repeated weights

Example 1

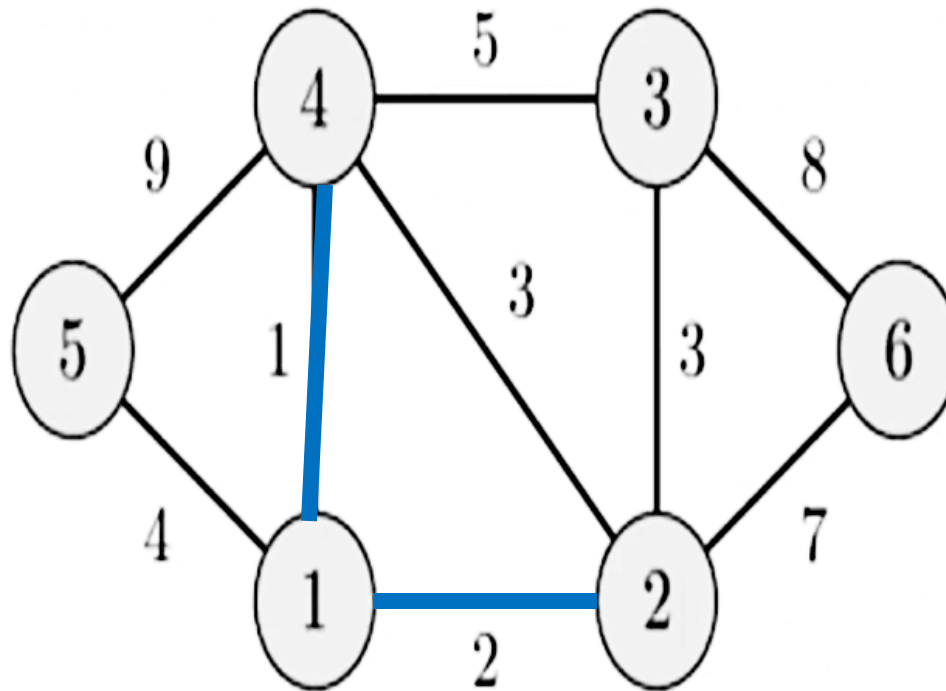


Iter 1



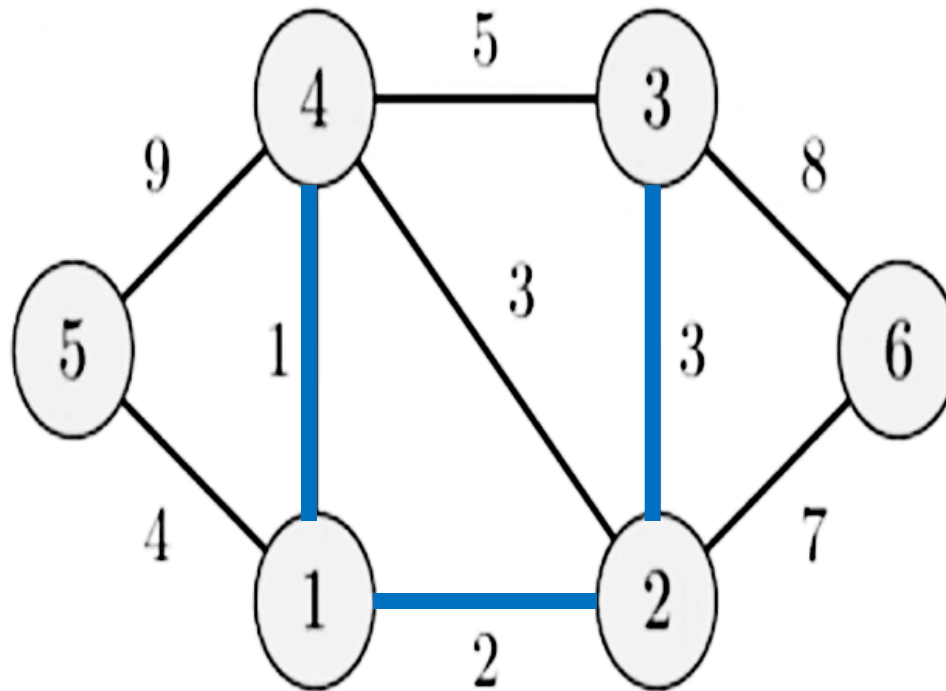
Start with the edge with min weight Select edge 1-4

Iter 2



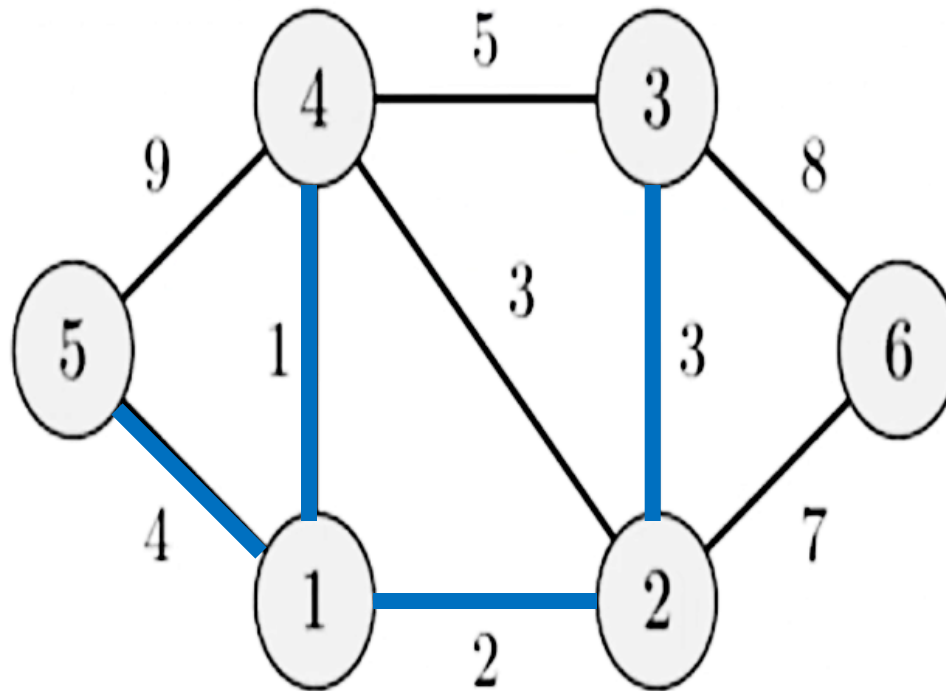
Select edge 1-2

Iter 3



Select edge 2-3

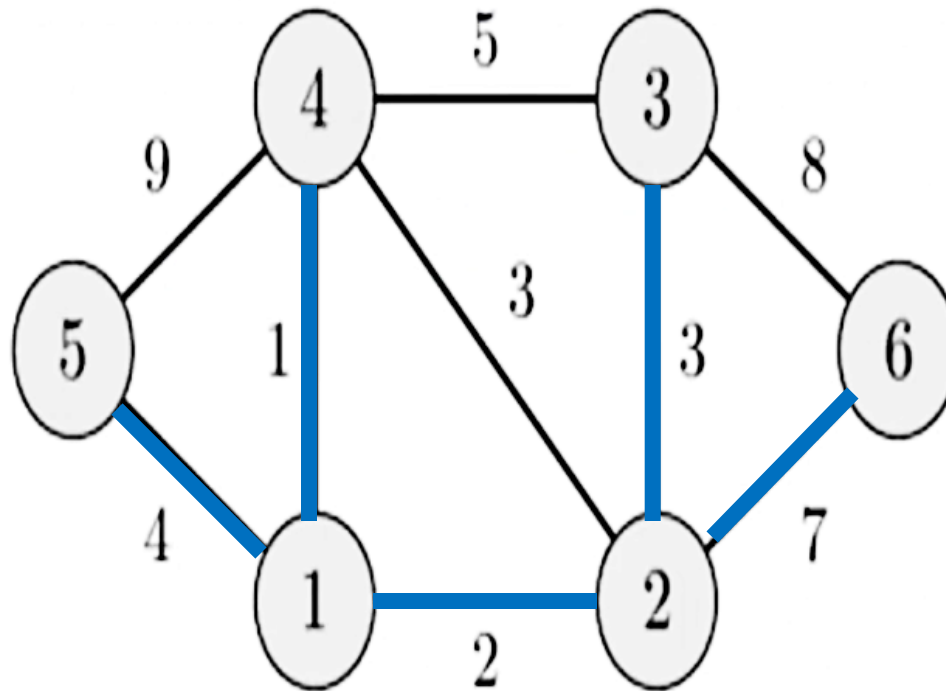
Iter 4



Select edge 1- 5

Iter 5

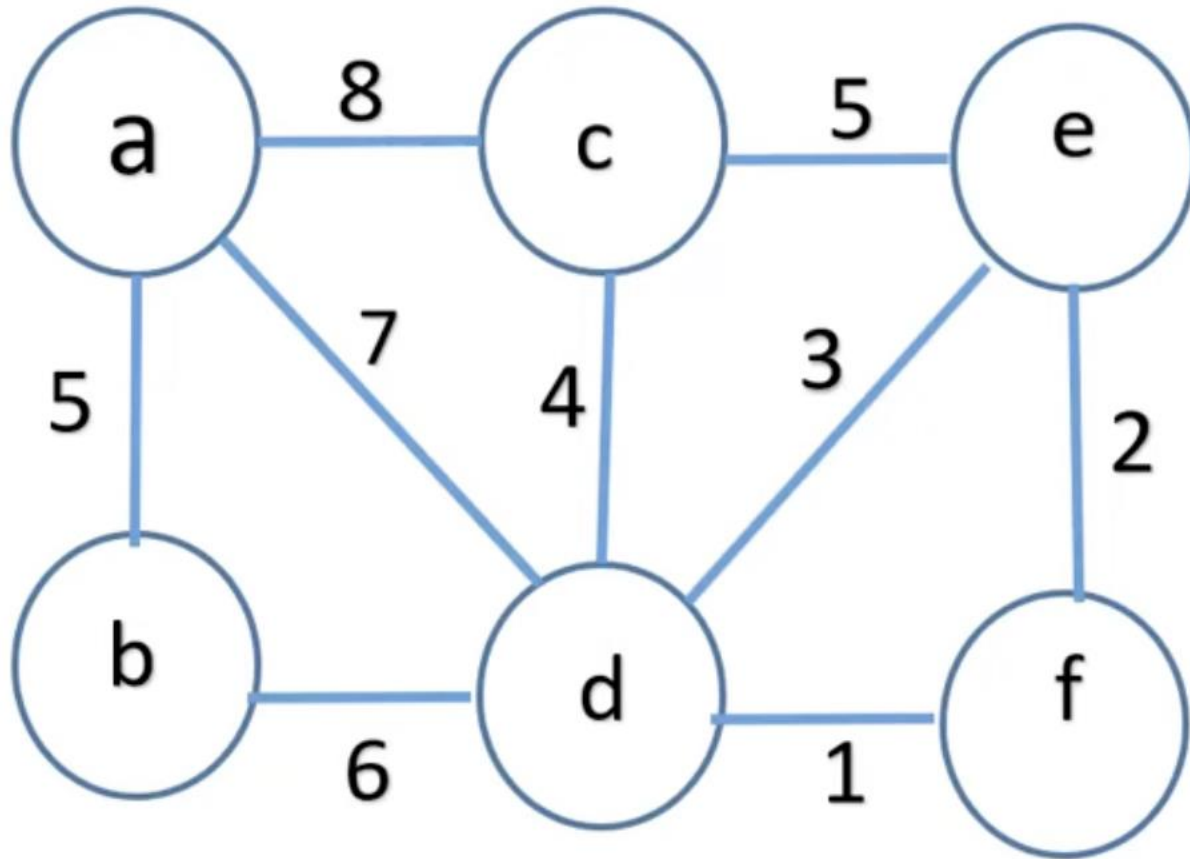
Number of iterations is $n-1$ vertices



Select edge 2- 6

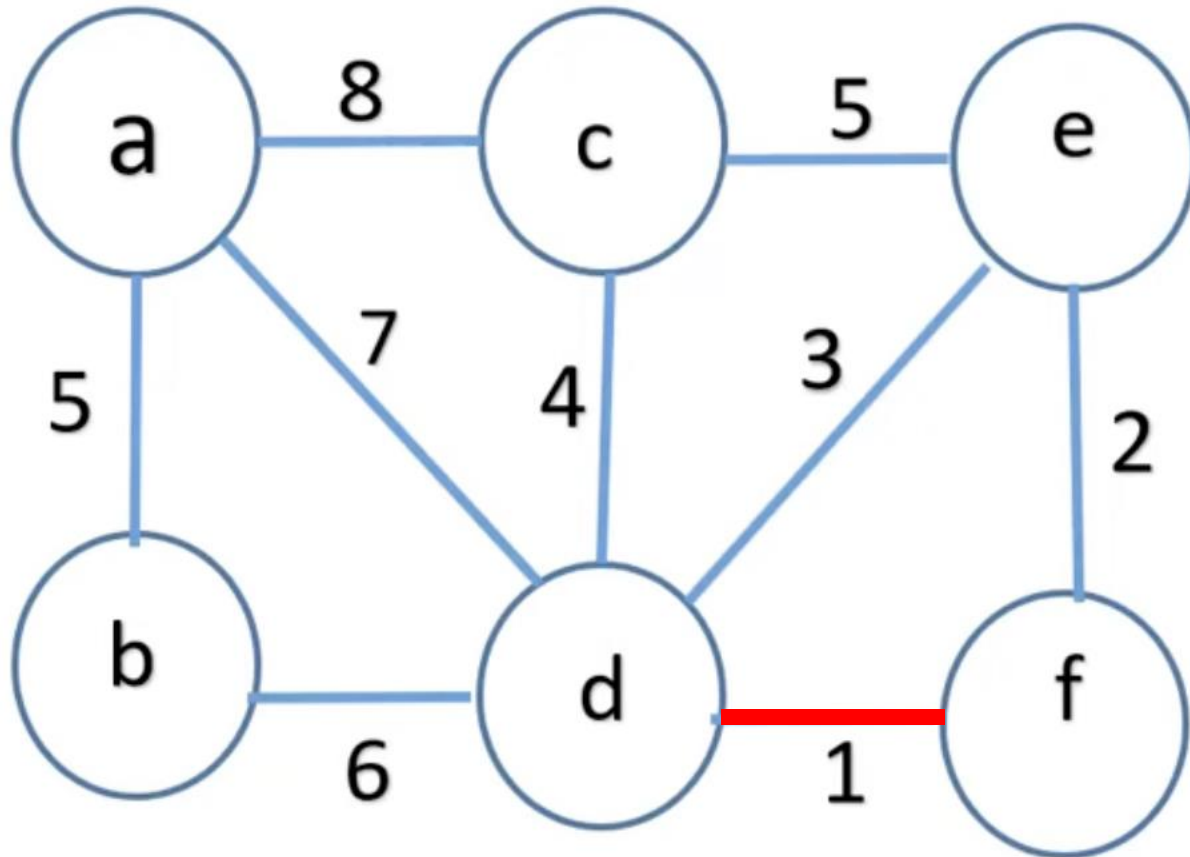
$$\text{Cost} = 1 + 4 + 2 + 3 + 7 = 18$$

Example 2



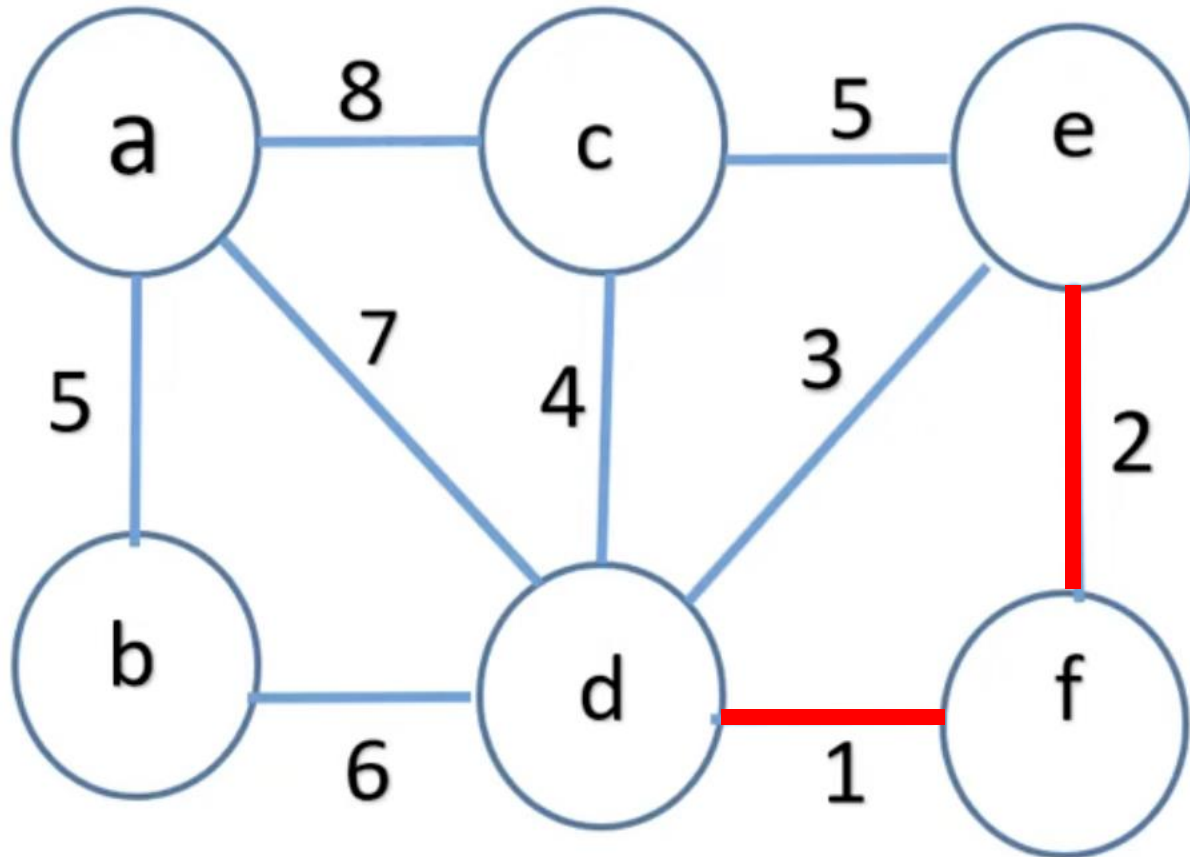
Using Kruskal algorithm, find the MST

Iteration 1



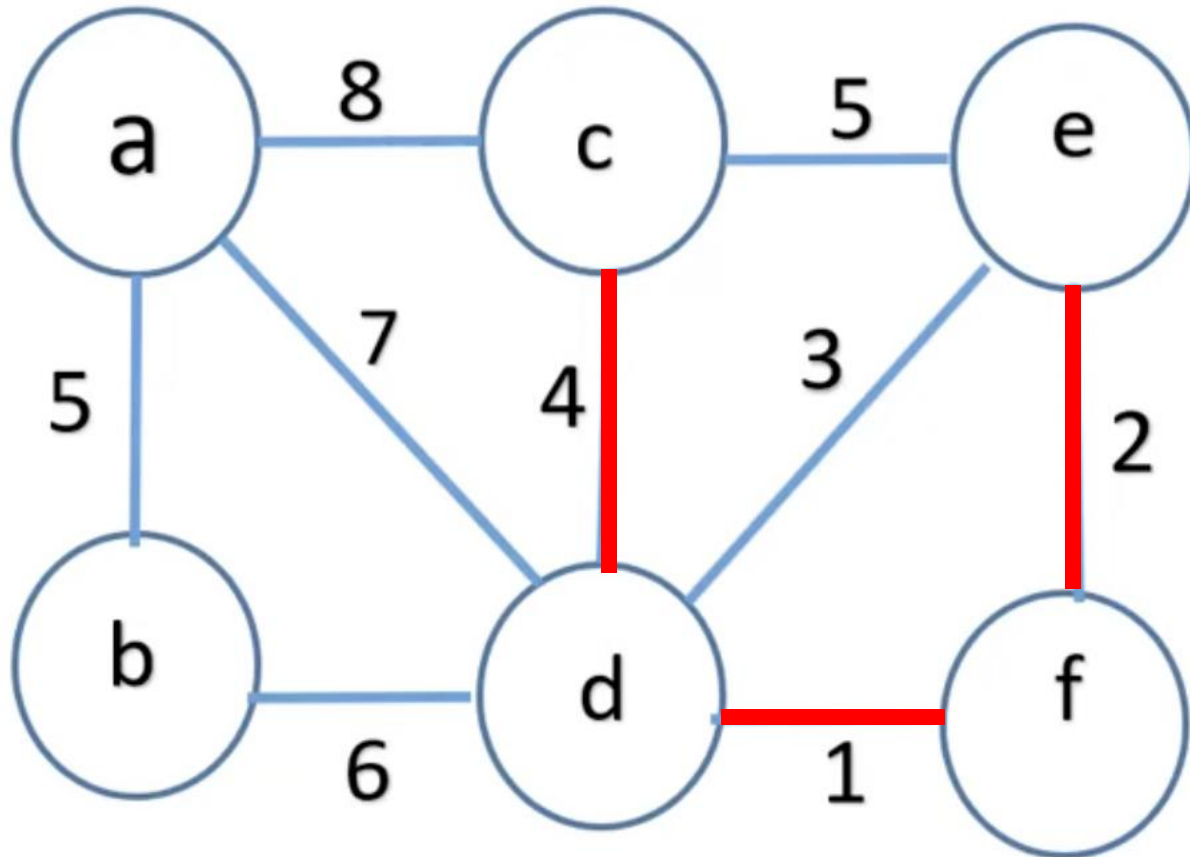
Select edge d-f

Iteration 2



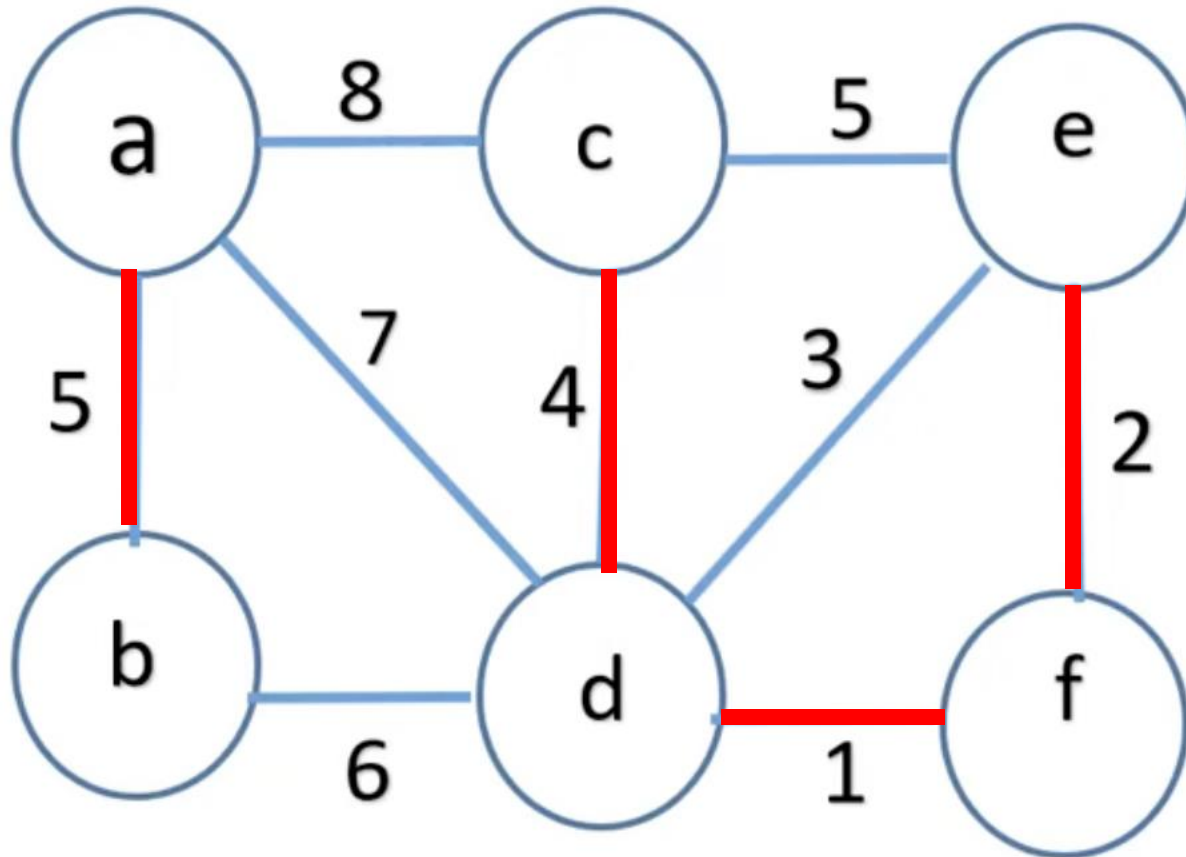
Select edge f-e

Iteration 3



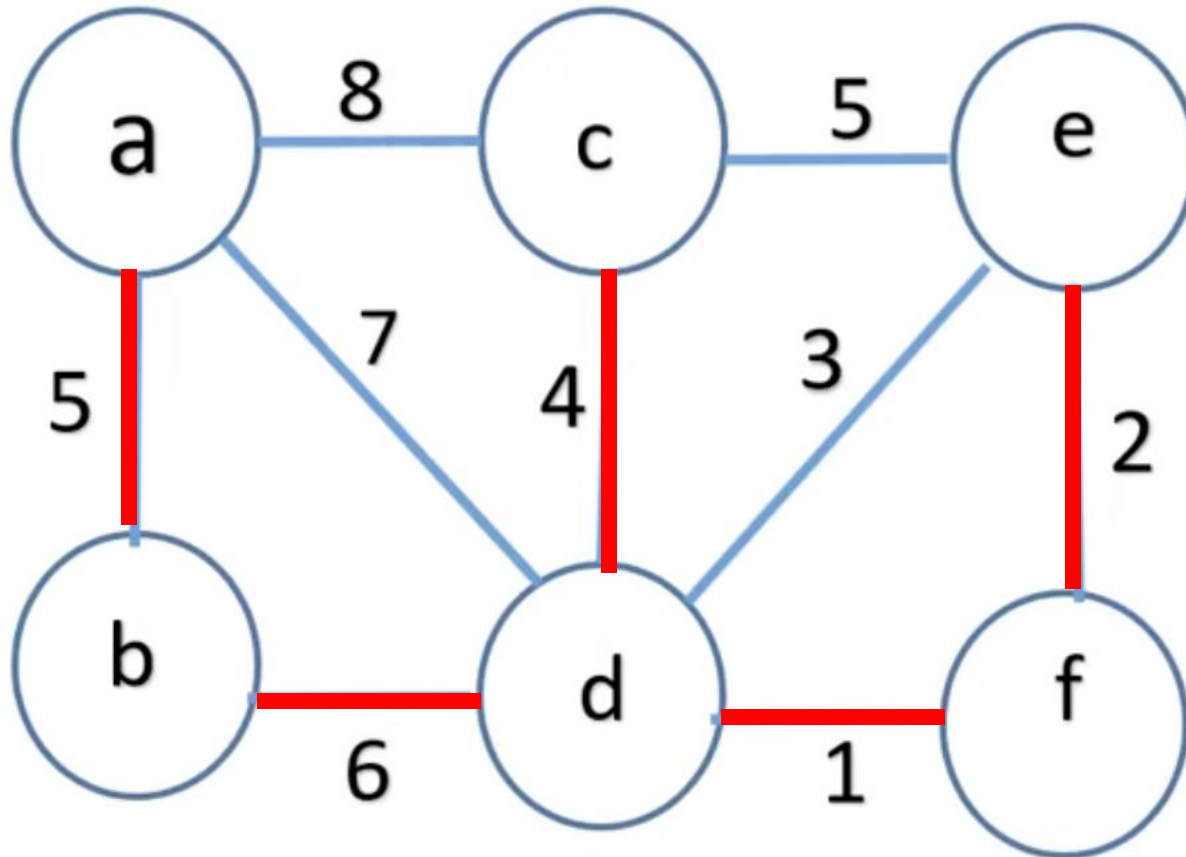
Select edge c-d

Iteration 4



Select edge a-b

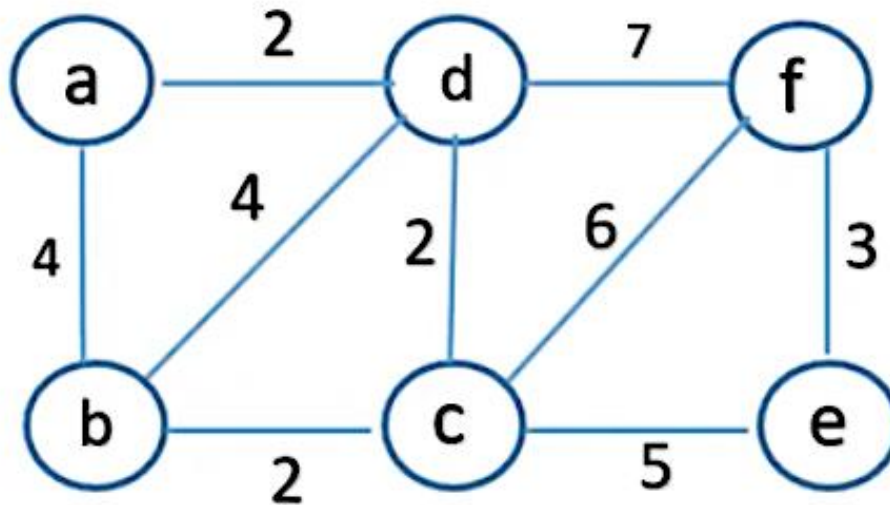
Iteration 5



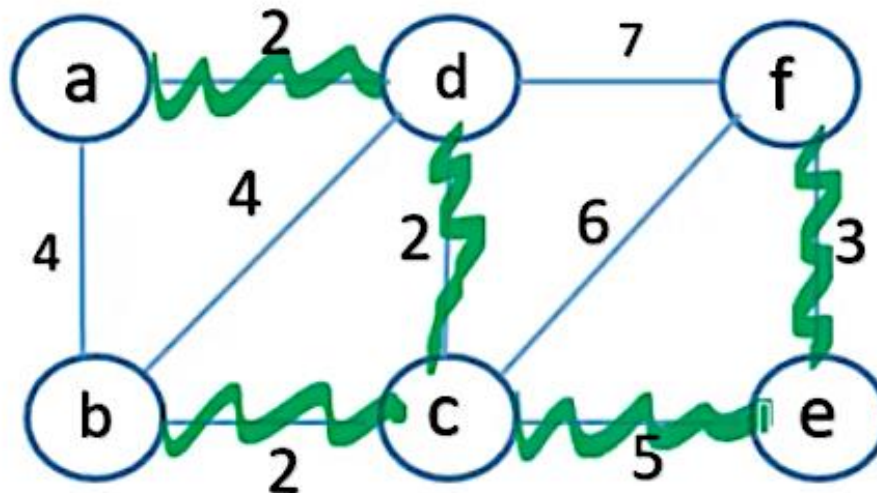
Select edge b-d

$$\text{Cost} = 5 + 6 + 4 + 1 + 2 = 18$$

Example 3

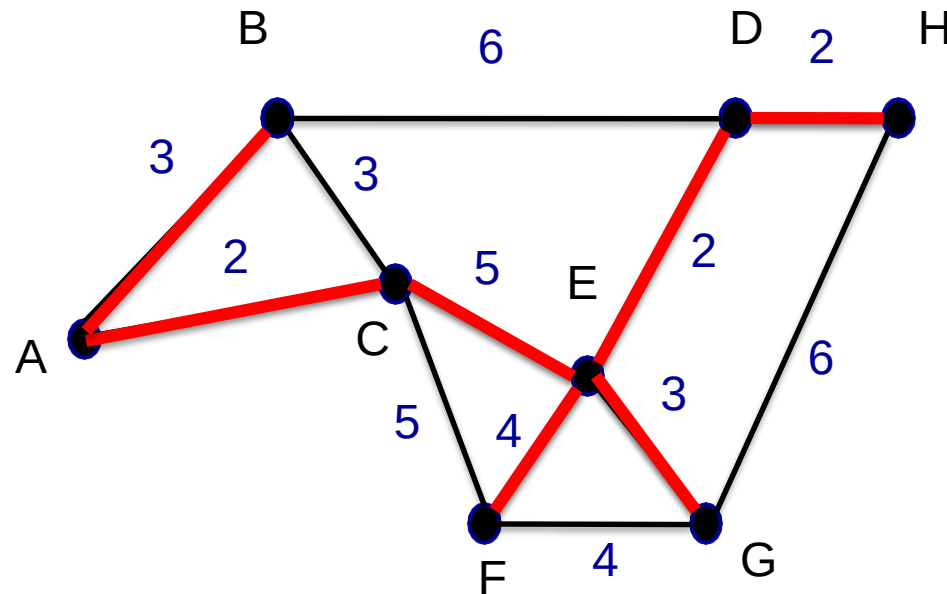


Sol.



Minimal Spanning Trees

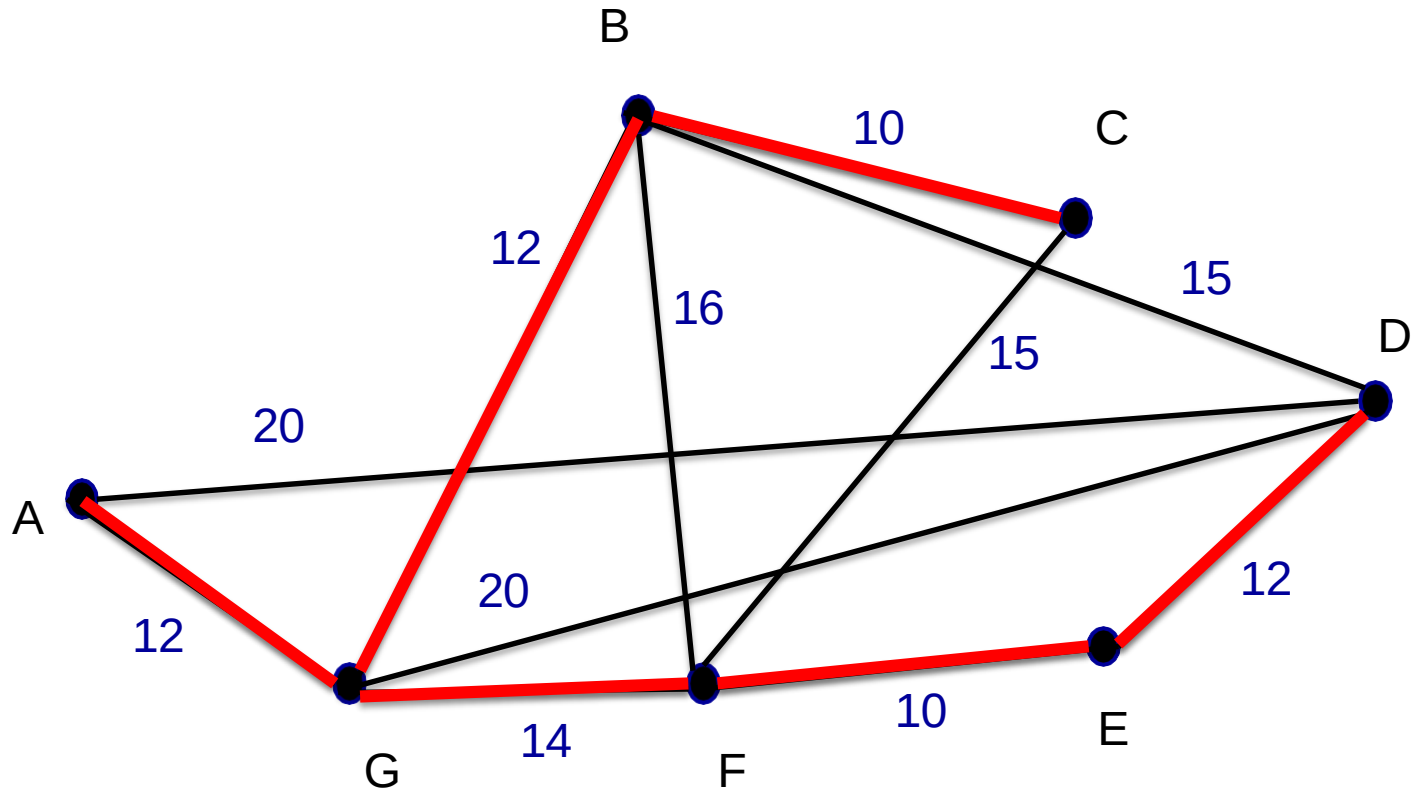
■ Example 4



21 Kilometers

Minimal Spanning Trees

■ Example 5



$$\text{Cost} = 12 + 14 + 10 + 12 + 12 + 10 = 68$$

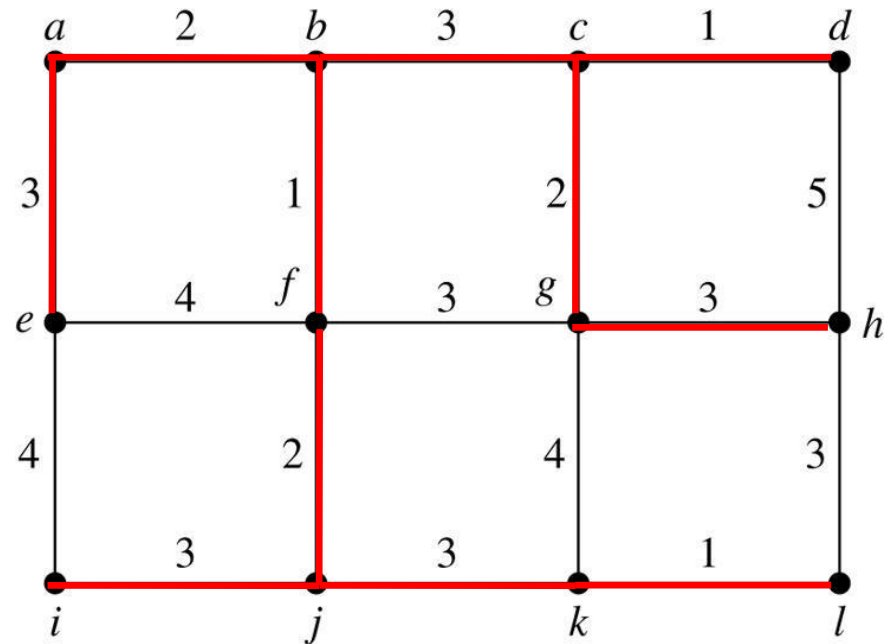
Sol.

Total: 24



T#2 Use Kruskal algorithm to find a minimum spanning tree of G .

Choice	Edge	Weight
1	$\{c, d\}$	1
2	$\{k, l\}$	1
3	$\{b, f\}$	1
4	$\{c, g\}$	2
5	$\{a, b\}$	2
6	$\{f, j\}$	2
7	$\{b, c\}$	3
8	$\{j, k\}$	3
9	$\{g, h\}$	3
10	$\{i, j\}$	3
11	$\{a, e\}$	3
Total:		24



Process tree will typically have several